



Dependabilidade

Conceitos Básicos e Taxonomia

Um guia de leitura do artigo

"Basic Concepts and Taxonomy of Dependable and Secure Computing"

Avizienis, Laprie, Randell & Landwehr

IEEE Transactions on Dependable and Secure Computing, v. 1, n. 1, 2004

Prof. Dr. Iaçanã Ianiski Weber

Confiabilidade e Segurança de Software

- 1 Por que este artigo
- 2 Conceitos básicos: sistema, serviço e as três ameaças
- 3 Ameaças I: a taxonomia das faltas
- 4 Ameaças II: falhas, erros e a patologia da falha
- 5 Atributos revisitados e conceitos vizinhos
- 6 Os meios: prevenção, tolerância, remoção e previsão
- 7 Síntese e guia de leitura

- 1 Por que este artigo
- 2 Conceitos básicos: sistema, serviço e as três ameaças
- 3 Ameaças I: a taxonomia das faltas
- 4 Ameaças II: falhas, erros e a patologia da falha
- 5 Atributos revisitados e conceitos vizinhos
- 6 Os meios: prevenção, tolerância, remoção e previsão
- 7 Síntese e guia de leitura

O contrato desta aula

Vamos percorrer um artigo inteiro, seção por seção. Ao final você deve conseguir:

- distinguir **falta**, **erro** e **falha** (*fault*, *error*, *failure*) e explicar como um vira o outro;
- classificar qualquer falta nas **oito dimensões** da taxonomia;
- nomear os **seis atributos** de dependabilidade e dizer qual deles um incidente violou;
- escolher entre os **quatro meios** (prevenção, tolerância, remoção, previsão de faltas) com justificativa;
- ler o artigo sozinho, sem travar no vocabulário.

A ficha técnica do artigo

- Publicado no **volume 1, número 1** da **IEEE TDSC** (jan-mar 2004), p. 11-33. É o artigo que abre o periódico.
- TDSC**: *IEEE Transactions on Dependable and Secure Computing*, o periódico da IEEE Computer Society dedicado a dependabilidade e segurança. Repare que o nome do periódico já carrega a tese do artigo: os dois campos tratados juntos.
- Não é um artigo de resultados. É um artigo de **vocabulário**: propõe um conjunto de conceitos e as relações entre eles.
- Os autores declaram dois públicos-alvo: **organizações de padronização e ensino**.

Como o artigo se define

"Uma tentativa de documentar um consenso mínimo sobre conceitos dentro de várias especialidades, para facilitar interações técnicas fecundas."

Aviso dos próprios autores

O artigo não documenta o estado da arte e não trata de implementação. Ele trata de **conceitos**.

Avizienis et al. (2004), §1

Vinte e quatro anos de discussão antes da primeira linha

A taxonomia não nasceu deste artigo. Ela é o resultado de um comitê:

- **1980:** comitê conjunto “Fundamental Concepts and Terminology”, formado pelo TC on Fault-Tolerant Computing (IEEE CS) e pelo IFIP WG 10.4.
- **1982:** sete artigos de posição numa sessão especial do FTCS-12.
- **1985:** síntese apresentada no FTCS-15 por Laprie, antecessora direta deste artigo.
- **1992:** o livro *Dependability: Basic Concepts and Terminology*. Nesta etapa entram **segurança** como atributo e as **faltas maliciosas** na taxonomia.
- **1995 em diante:** esforço contínuo de expandir, refinar e simplificar, que resulta neste artigo de 2004.

Quando um vocabulário leva 24 anos para estabilizar, o problema não era achar palavras bonitas. Era decidir **quais distinções valem a pena fazer**.

Avizienis et al. (2004), §1, “Preceding Work and Goals for the Future”

Quem são os quatro autores

Autor	Instituição (2004)	O que traz para a mesa
Algirdas Avizienis	UCLA / Vytautas Magnus	Projeto de computadores tolerantes a faltas desde 1967; programação em N versões e diversidade de projeto; taxonomia de faltas de projeto em microprocessadores comerciais.
Jean-Claude Laprie	LAAS-CNRS, Toulouse	Autor da síntese de 1985 e editor do livro de 1992; responsável por consolidar <i>dependability</i> como conceito guarda-chuva.
Brian Randell	Univ. de Newcastle	Tradição de tolerância a faltas em software e recuperação por retorno a estado salvo.
Carl Landwehr	Univ. de Maryland / NSF	Vem da comunidade de segurança : autor de uma taxonomia de falhas de segurança em programas (1994), base das definições de lógica maliciosa deste artigo.

Três autores vêm de **tolerância a faltas** e um vem de **segurança**. Essa composição explica a tese central: os dois campos estavam resolvendo o mesmo problema com nomes diferentes.

A tese: dois campos convergindo

O artigo abre com um diagnóstico histórico.

Dependabilidade percebeu que...

...restringir-se a faltas **não maliciosas** resolvia só parte do problema. Um sistema que tolera raio cósmico mas não tolera um invasor não é dependável.

Segurança percebeu que...

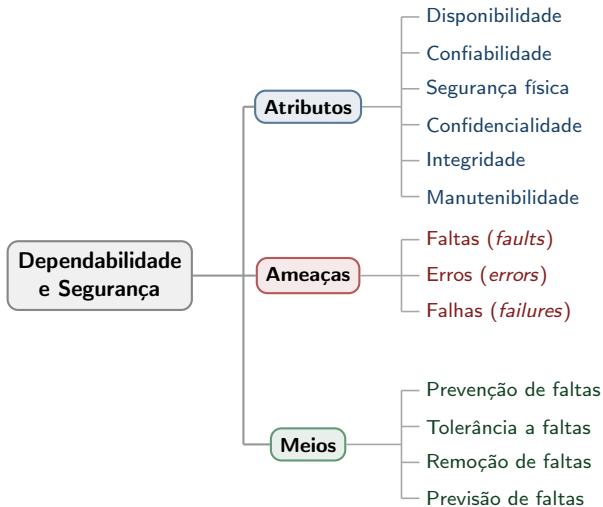
...o foco histórico em **confidencialidade** precisava ser acompanhado de integridade e disponibilidade. Elas sempre estiveram nas definições, mas recebiam menos atenção.

Um ataque, na linguagem do artigo, é apenas uma **falta externa maliciosa**. Nada mais. Isso permite tratar um invasor com o mesmo ferramental que trata um bit flip.

Os autores reconhecem uma limitação: por restrição de espaço, confidencialidade não recebe no artigo a atenção que merece.

Avizienis et al. (2004), §1 e §3.5

O mapa da aula: a árvore da dependabilidade



Três perguntas, três ramos. **Atributos:** o que eu quero do sistema? **Ameaças:** o que pode dar errado? **Meios:** como eu chego lá? A aula segue exatamente esta ordem.

- 1 Por que este artigo
- 2 Conceitos básicos: sistema, serviço e as três ameaças
- 3 Ameaças I: a taxonomia das faltas
- 4 Ameaças II: falhas, erros e a patologia da falha
- 5 Atributos revisitados e conceitos vizinhos
- 6 Os meios: prevenção, tolerância, remoção e previsão
- 7 Síntese e guia de leitura

Sistema, ambiente, fronteira

Definição do paper

Um **sistema** (*system*) é uma entidade que interage com outras entidades: outros sistemas, hardware, software, humanos e o mundo físico. Essas outras entidades são o **ambiente** (*environment*). A **fronteira do sistema** (*system boundary*) é a divisa comum entre os dois.

- A definição é deliberadamente ampla: cobre de uma porta lógica a uma rede de computadores com operadores humanos.
- O artigo distingue **sistemas de computação e comunicação** de **sistemas baseados em computador** (*computer-based systems*), que incluem também os humanos e as organizações ao redor.
- Propriedades fundamentais de um sistema: **funcionalidade, desempenho, dependabilidade e segurança, custo.**

Observe que

Quase toda confusão nesta área vem de **não declarar onde está a fronteira**. A mesma ocorrência é falta interna para um observador e falta externa para outro, dependendo de onde ele desenhou a linha.

Estrutura: a recursão para e onde você mandar

- Um sistema é composto de **componentes** (*components*) ligados para interagir. Cada componente é, ele mesmo, outro sistema.
- A recursão para quando um componente é considerado **atômico** (*atomic*): sua estrutura interna não pode ser discernida, ou não interessa e pode ser ignorada.
- Consequência direta: o **estado total** do sistema é o conjunto dos estados externos de seus componentes atômicos.

Na prática

“Atômico” é uma decisão de análise, não um fato da natureza. Ao analisar um serviço, você pode declarar o banco de dados atômico. Para o time do banco de dados, ele é um sistema com dezenas de componentes.

A taxonomia é **recursiva e relativa ao observador**. Isso não é imprecisão: é o que a torna aplicável de uma porta lógica a uma constelação de satélites.

Avizienis et al. (2004), §2.1

Quatro palavras que não são sinônimos

Função (<i>function</i>)	O que o sistema deveria fazer. Descrita pela especificação funcional , em termos de funcionalidade e desempenho.
Comportamento (<i>behavior</i>)	O que o sistema faz para implementar a função. Descrito por uma sequência de estados.
Estrutura (<i>structure</i>)	O que permite ao sistema gerar o comportamento. Os componentes e suas ligações.
Serviço (<i>service</i>)	O comportamento tal como percebido pelo usuário .

Armadilha

Comportamento e serviço não são a mesma coisa. O comportamento inclui tudo o que acontece dentro. O serviço é só a parte que atravessa a interface e chega ao usuário. É por isso que um sistema pode estar internamente errado e ainda assim entregar serviço correto.

Avizienis et al. (2004), §2.1

Estado total, estado externo, estado interno

Definição do paper

O **estado total** (*total state*) de um sistema é o conjunto de: computação, comunicação, informação armazenada, interconexão e condição física.

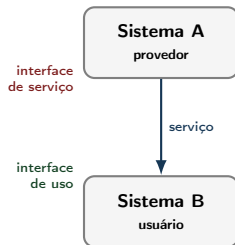
- A parte do estado total **perceptível na interface de serviço** é o **estado externo** (*external state*).
- O resto é o **estado interno** (*internal state*).
- O **serviço entregue** é a sequência dos estados externos do provedor.

Repare que “condição física” está no estado. Temperatura, tensão e desgaste fazem parte do estado do sistema, não são detalhes de outra disciplina.

Avizienis et al. (2004), §2.1

Provedor, usuário e as duas interfaces

- O **provedor** (*provider*) entrega serviço. O **usuário** (*user*) é outro sistema que recebe esse serviço.
- A parte da fronteira do provedor onde a entrega ocorre é a **interface de serviço** (*service interface*).
- A interface do usuário onde ele recebe o serviço é a **interface de uso** (*use interface*).
- Um mesmo sistema pode ser provedor e usuário, simultaneamente ou em sequência.



Na prática

Um sistema geralmente implementa mais de uma função e entrega mais de um serviço. O artigo chama essas partes de **itens de função** e **itens de serviço**, e usa o plural quando precisa distingui-los.

Serviço correto e falha de serviço

Definição do paper

Serviço correto (*correct service*) é entregue quando o serviço implementa a função do sistema. Uma **falha de serviço** (*service failure*) é um **evento** que ocorre quando o serviço entregue desvia do serviço correto.

- A falha é a **transição** de serviço correto para serviço incorreto.
- O período de entrega de serviço incorreto é a **interrupção** (*service outage*).
- A transição de volta é a **restauração** (*service restoration*).

Armadilha

Falha é **evento** (instantâneo), interrupção é **período** (tem duração). Confundir os dois estraga qualquer conversa sobre disponibilidade, porque disponibilidade se mede em duração, não em contagem de eventos.

Avizienis et al. (2004), §2.2

Duas maneiras de um serviço falhar

O artigo faz aqui uma distinção que muita gente ignora a vida inteira:

- 1 O serviço **não cumpre a especificação funcional**. Caso comum, e o que todo mundo chama de bug.
- 2 O serviço **cumpe a especificação**, mas a **especificação não descrevia adequadamente a função** do sistema.

Consequência formal

A ocorrência de uma falha é definida em relação à **função** do sistema, não em relação à descrição dessa função na especificação. Uma entrega que cumpre a especificação pode ser inaceitável para o usuário, revelando uma **falta de especificação** (*specification fault*).

Por isso o artigo admite que falhas podem ser **subjetivas e discutíveis**: às vezes só se reconhece que houve falha depois, pelas consequências.

Avizienis et al. (2004), §2.2 e §3.3.1

Erro e falta: as outras duas ameaças

Definição do paper

Erro (*error*): a parte do estado total do sistema que pode levar a uma falha de serviço subsequente.

Falta (*fault*): a causa **julgada ou hipotetizada** de um erro.

- “Julgada ou hipotetizada” (*adjudged or hypothesized*) não é enfeite. A falta é o resultado de um **diagnóstico**, e diagnósticos podem estar errados.
- Uma falta é **ativa** (*active*) quando causa um erro; caso contrário é **dormente** (*dormant*).
- Faltas podem ser **internas** ou **externas** ao sistema.
- Na maioria dos casos a falta primeiro causa erro no estado de um **componente**, e o estado externo do sistema não é afetado de imediato.

Muitos erros nunca alcançam o estado externo e nunca causam falha. Guarde essa frase: ela sustenta metade da §5.

Vulnerabilidade: a ponte entre o externo e o interno

Definição do paper

Uma **vulnerabilidade** (*vulnerability*) é uma **falta interna** que permite que uma **falta externa** cause dano ao sistema.

- A presença prévia de uma vulnerabilidade é **necessária** para que uma falta externa cause erro e, possivelmente, falha.
- Vulnerabilidades podem ser faltas de desenvolvimento ou operacionais; podem ser maliciosas ou não maliciosas.
- Uma vulnerabilidade pode resultar de uma **falta deliberada de desenvolvimento**, por razões econômicas ou de usabilidade, que resulta em proteção limitada ou ausente.

Na prática

O artigo aponta a analogia: há semelhança entre uma tentativa de intrusão e uma falta física externa que “explora” a falta de blindagem. Nos dois casos, sem a vulnerabilidade interna, o agente externo não produz efeito.

Falha parcial e modo degradado

- Quando a especificação funcional inclui várias funções, a falha de uma delas pode deixar o sistema em **modo degradado** (*degraded mode*), ainda oferecendo um subconjunto útil de serviços.
- A própria especificação pode identificar esses modos: serviço lento, serviço limitado, serviço de emergência.
- Nesse caso o sistema sofreu uma **falha parcial** (*partial failure*) de funcionalidade ou de desempenho.

Modo degradado só é modo degradado se estiver **na especificação**. Se não estiver, é comportamento incorreto que alguém resolveu chamar de degradação depois do fato.

Na prática

Falhas de desenvolvimento e falhas de dependabilidade, que veremos na §3.3, também podem ser parciais.

Avizienis et al. (2004), §2.2

Dependabilidade: duas definições, dois usos

Definição original

Dependabilidade é a capacidade de entregar serviço em que se possa **justificadamente** confiar.

Definição alternativa

Dependabilidade é a capacidade de um sistema de **evitar falhas de serviço mais frequentes ou mais severas do que é aceitável**.

- A primeira **generaliza** as noções clássicas (confiabilidade, disponibilidade, segurança...), que viram atributos.
- A segunda fornece um **critério de decisão**: um sistema falha e mesmo assim continua dependável; a partir de quando deixa de ser?

Armadilha

A palavra “justificadamente” carrega o artigo inteiro. Confiança sem evidência é otimismo, não dependabilidade. Remoção e previsão de faltas existem para produzir essa justificativa.

Dependência e confiança

Definição do paper

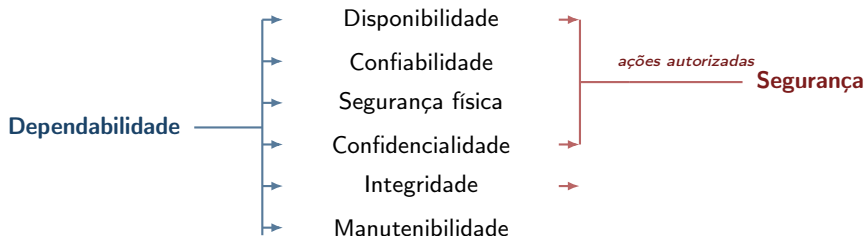
A **dependência** (*dependence*) do sistema A em relação ao sistema B é a extensão em que a dependabilidade de A é (ou seria) afetada pela de B.

Confiança (*trust*) é **dependência aceita**.

- A dependência varia de **total** (qualquer falha de B faz A falhar) a **independência completa** (B não consegue fazer A falhar).
- Se a dependabilidade de B for insuficiente para o que A precisa, há três saídas: melhorar B, reduzir a dependência de A, ou acrescentar tolerância a faltas em A.
- A aceitação pode ser explícita (contrato), implícita, irrefletida, ou até involuntária, quando A não tem alternativa.

A medida operacional proposta pelo artigo é desconfortável e certa: **a extensão em que A deixa de prover meios de tolerar as falhas de B é a medida da confiança de A em B.**

Os seis atributos



Dependabilidade e segurança não são conjuntos disjuntos nem um contém o outro. Elas **compartilham** disponibilidade e integridade, e cada uma tem ênfases próprias.

Avizienis et al. (2004), Fig. 1 e Fig. 14, §2.3 e §4.1

Os seis atributos, em uma linha cada

Disponibilidade (<i>availability</i>)	Prontidão para serviço correto.
Confiabilidade (<i>reliability</i>)	Continuidade de serviço correto.
Segurança física (<i>safety</i>)	Ausência de consequências catastróficas sobre os usuários e o ambiente.
Integridade (<i>integrity</i>)	Ausência de alterações impróprias do sistema.
Manutenibilidade (<i>maintainability</i>)	Capacidade de sofrer modificações e reparos.
Confidencialidade (<i>confidentiality</i>)	Ausência de divulgação não autorizada de informação.

Disponibilidade não é confiabilidade

Prontidão versus **continuidade**. Um sistema que cai por 1 segundo a cada minuto tem disponibilidade de 98,3% e confiabilidade sofrível para qualquer tarefa que dure mais de um minuto. Um sistema que cai uma vez por ano por 10 horas tem confiabilidade alta e disponibilidade pior em termos de duração total.

A confidencialidade entra separada porque é o atributo que ganha destaque quando se trata de **segurança**.

Segurança é um composto, não um atributo

Definição do paper

Segurança (*security*) é um composto dos atributos de **confidencialidade**, **integridade** e **disponibilidade**, exigindo a existência concomitante de:

- 1 disponibilidade **apenas para ações autorizadas**;
- 2 confidencialidade;
- 3 integridade, com “impróprio” significando “não autorizado”.

- Na §4.1 os autores oferecem uma definição unificada: segurança é a **ausência de acesso ou manipulação não autorizada do estado do sistema**.
- Repare no mecanismo: os mesmos atributos, restritos pela noção de **autorização**, viram segurança.

Isso resolve elegantemente a triáde CIA: ela não é uma lista arbitrária, é o que sobra dos atributos de dependabilidade quando você acrescenta a palavra “autorizado”.

Avizienis et al. (2004), §2.3 e §4.1

A especificação de dependabilidade e segurança

Definição do paper

A **especificação de dependabilidade e segurança** deve incluir os requisitos para os atributos em termos da **frequência e severidade aceitáveis** de falhas de serviço, para **classes de faltas especificadas** e um **ambiente de uso dado**.

- Um ou mais atributos podem simplesmente **não ser exigidos** para um dado sistema.
- Essa especificação também pode conter faltas: omitir classes de faltas relevantes, descrever mal o ambiente de uso, ou exigir requisitos altos demais sem justificativa, o que encarece o desenvolvimento.

Exemplo do próprio artigo

No sistema AAS da aviação civil norte-americana, o limite inicial de indisponibilidade total era de **3 segundos por ano**. Em 1994 ele foi renegociado para **5 minutos por ano** no novo contrato. Um requisito mal calibrado é, ele próprio, uma falta.

Avizienis et al. (2004), §2.3 e §3.3.3

Checkpoint 1

Classifique, com a taxonomia da §2

Um serviço web mantém um contador de sessões ativas. Por um erro de arredondamento, o contador guarda 41 quando o valor correto seria 42. O contador só é usado para dimensionar um *pool* interno, e nesse dia o *pool* tinha folga. Nenhum usuário percebeu qualquer diferença.

- 1 Houve **falta**? Houve **erro**? Houve **falha**?
- 2 O sistema esteve **dependável** durante esse período?

Checkpoint 1

Classifique, com a taxonomia da §2

Um serviço web mantém um contador de sessões ativas. Por um erro de arredondamento, o contador guarda 41 quando o valor correto seria 42. O contador só é usado para dimensionar um *pool* interno, e nesse dia o *pool* tinha folga. Nenhum usuário percebeu qualquer diferença.

- 1 Houve **falta**? Houve **erro**? Houve **falha**?
- 2 O sistema esteve **dependável** durante esse período?

Falta: sim, o defeito no código de arredondamento (dormente até ser ativado).

Erro: sim, o contador com 41 é parte do estado total que **poderia** levar a falha.

Falha: não. O erro nunca alcançou o estado externo. Não houve desvio do serviço.

Dependável: sim, pelo critério da segunda definição, porque não houve falha de serviço. Mas o erro latente é risco acumulado, e o sistema chegou perto.

- 1 Por que este artigo
- 2 Conceitos básicos: sistema, serviço e as três ameaças
- 3 Ameaças I: a taxonomia das faltas**
- 4 Ameaças II: falhas, erros e a patologia da falha
- 5 Atributos revisitados e conceitos vizinhos
- 6 Os meios: prevenção, tolerância, remoção e previsão
- 7 Síntese e guia de leitura

O ciclo de vida tem duas fases, e ambas produzem faltas

Fase de desenvolvimento

Da concepção inicial do usuário até a decisão de que o sistema passou em todos os testes de aceitação e está pronto para entregar serviço.

Aqui entram as **faltas de desenvolvimento** (*development faults*).

Fase de uso

Começa quando o sistema é aceito e passa a entregar serviço. Alterna três períodos: **entrega de serviço**, **interrupção** (*outage*) e **desligamento** (*shutdown*).

Aqui entram as **faltas operacionais** (*operational faults*).

- A interrupção é causada por uma falha. O **desligamento** é uma parada **intencional** por entidade autorizada.
- Ações de manutenção podem ocorrer em qualquer um dos três períodos.

Avizienis et al. (2004), §3.1

Os dois ambientes, elemento por elemento

Ambiente de desenvolvimento

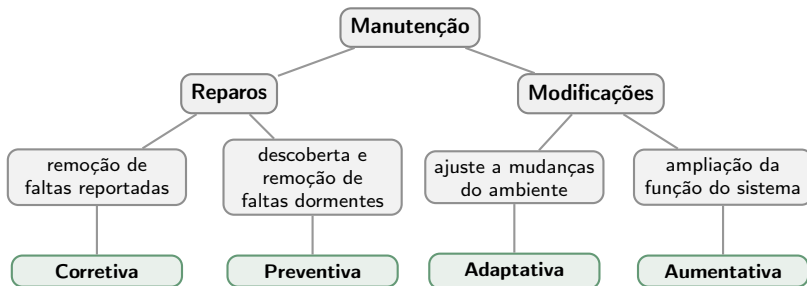
- 1 o mundo físico e seus fenômenos naturais;
- 2 **desenvolvedores humanos**, alguns possivelmente sem competência ou com objetivos maliciosos;
- 3 **ferramentas de desenvolvimento**, software e hardware;
- 4 **instalações de produção e teste**.

Ambiente de uso

- 1 o mundo físico e seus fenômenos naturais;
- 2 **administradores** (inclusive mantenedores), que têm autoridade para gerenciar, modificar, reparar e usar; alguns podem não ter competência ou ter objetivos maliciosos;
- 3 **usuários**, que recebem serviço;
- 4 **provedores**, que entregam serviço ao sistema;
- 5 **infraestrutura**: fontes de informação (hora, GPS), enlaces de comunicação, energia, refrigeração;
- 6 **intrusos**: *hackers*, vândalos, insiders corruptos, agentes de governos ou organizações hostis, e software malicioso.

O intruso é listado como **elemento normal do ambiente de uso**, ao lado da refrigeração e do relógio. É assim que segurança entra na taxonomia sem precisar de um arcabouço separado.

Manutenção é um processo de desenvolvimento



“Manutenção” aqui inclui não apenas reparos, mas **todas** as modificações do sistema durante a fase de uso. Portanto, manutenção é desenvolvimento, e tudo o que vale para desenvolvimento vale para ela.

Reparo x tolerância

A distinção é uma só: **manutenção exige um agente externo** (técnico, equipamento de teste, recarga remota de software). Tolerância a faltas é feita pelo próprio sistema.

Repare na recursão: reparo pode ser visto como tolerância a faltas de um sistema maior, que inclui o sistema reparado mais as pessoas que o reparam.

As oito dimensões de classificação de faltas



Toda falta que existe pode ser posicionada nessas oito dimensões. Este é o instrumento central do artigo, e o que você vai usar nos exercícios.

Avizienis et al. (2004), Fig. 4, §3.2.1

De 256 possibilidades para 31 classes plausíveis

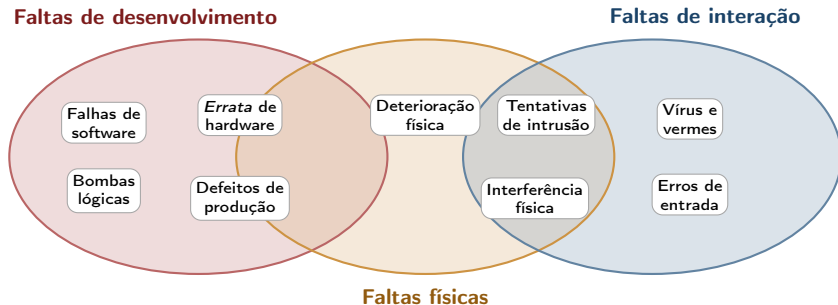
- Oito dimensões binárias dariam $2^8 = 256$ combinações.
- Nem todo critério se aplica a toda classe. Faltas naturais, por exemplo, não podem ser classificadas por objetivo, intenção e capacidade: não há humano envolvido.
- Os autores identificam **31 combinações prováveis**, e dizem que outras podem ser identificadas no futuro.
- Essas 31 classes se agrupam em **três conjuntos parcialmente sobrepostos**.

Por que isso importa

Conhecer as classes possíveis permite ao projetista **decidir quais delas entram na especificação** de dependabilidade e segurança. Você não tolera faltas em abstrato: você tolera classes declaradas.

Avizienis et al. (2004), §3.2.1, Fig. 5

Os três grandes grupos



Os grupos se **sobrepõem**. Uma falta física pode ser de desenvolvimento (defeito de produção) e uma falta de interação pode ser física (interferência). Não são gavetas exclusivas.

Avizienis et al. (2004), Fig. 5, §3.2.1

Faltas naturais

Faltas físicas causadas por fenômenos naturais, **sem participação humana**.

- **Defeitos de produção** (*production defects*): faltas naturais que se originam durante o desenvolvimento.
- Durante a operação, as faltas naturais são:
 - **internas**, por processos naturais de deterioração física;
 - **externas**, por processos que se originam fora da fronteira e causam interferência física, seja penetrando a fronteira de hardware (radiação), seja entrando pelas interfaces de uso (transientes de alimentação, linhas de entrada ruidosas).

Armadilha

Humanos também causam faltas físicas. “Falta física” e “falta natural” não são sinônimos: a primeira é a **dimensão** afetada, a segunda é a **causa fenomenológica**.

Avižienis et al. (2004), §3.2.2

Faltas humanas: três cortes sucessivos

Primeiro por **objetivo**, depois por **intenção**, depois por **capacidade**.

Objetivo	Intenção	Capacidade
Não maliciosas	Não deliberadas (<i>mistakes</i>): ações não intencionais das quais a pessoa não tem consciência	Acidentais ou de incompetência
	Deliberadas (<i>bad decisions</i>): ações intencionais que estão erradas e causam faltas	Acidentais ou de incompetência
Maliciosas	Sempre deliberadas	Não se aplica

- Faltas de desenvolvimento deliberadas e não maliciosas resultam em geral de **compromissos**: preservar desempenho aceitável, facilitar o uso, ou considerações econômicas.
- Faltas de interação deliberadas e não maliciosas vêm de um operador tentando contornar uma situação imprevista, ou violando um procedimento sem perceber as consequências.
- Elas costumam ser **reconhecidas como faltas só depois** de um comportamento inaceitável, ou seja, depois da falha.

A categoria mais desconfortável: faltas de incompetência

Definição do paper

Faltas de incompetência (*incompetence faults*) resultam da falta de competência profissional do humano autorizado, ou da inadequação da organização de desenvolvimento.

- Os autores argumentam: nem todo erro e nem toda má decisão de pessoas não maliciosas é um **acidente**. Algumas vêm de quem não tinha competência para o trabalho que assumiu.
- “Uma taxonomia completa não deve ocultar essa causa de faltas.”
- Como se reconhece uma? Quando as consequências envolvem perdas econômicas, lesões ou mortes, decidir se houve má prática profissional costuma exigir **comissão de inquérito independente ou processo judicial**.
- Existe também a **incompetência organizacional**: a equipe ou a empresa inteira não tinha condições de fazer o trabalho. O exemplo dado é a falha completa de desenvolvimento do sistema AAS.

Esta é uma contribuição nova de 2004 sobre as versões anteriores da taxonomia. Ela existe para permitir falar de **competência** sem eufemismo.

Três fontes de faltas que engenheiros subestimam

Errata de hardware

Em hardware, especialmente em microprocessadores, faltas de desenvolvimento são descobertas **depois** de a produção começar. Elas se chamam *errata* e são listadas em atualizações de especificação, publicadas periodicamente durante toda a vida do processador.

Componentes de prateleira (OTS)

Componentes *off-the-shelf* chegam com faltas de desenvolvimento conhecidas e podem conter faltas desconhecidas. As especificações deles podem ser incompletas ou incorretas. O problema é pior com componentes **legados**, que precisam ser mantidos por exigência do usuário.

Envelhecimento de software (*software aging*)

Condições de erro acumuladas progressivamente que resultam em degradação de desempenho ou falha completa. Exemplos do artigo: inchaço e vazamento de memória, *threads* não encerradas, travas de arquivo não liberadas, corrupção de dados, fragmentação de armazenamento e acúmulo de erros de arredondamento.

Algumas faltas de desenvolvimento existem porque as próprias ferramentas usadas são faltosas.

Avizienis et al.

(2004), §3.2.3

Faltas maliciosas: duas classes

Introduzidas com o objetivo de alterar o funcionamento do sistema durante o uso. Por causa do objetivo, classificar por **intenção** e **capacidade** não se aplica. Os alvos são três: interromper ou parar o serviço (negação de serviço), acessar informação confidencial, ou modificar o sistema indevidamente.

1. Lógica maliciosa

Abrange faltas de **desenvolvimento** (cavalo de Troia, bomba lógica, alça-pão) e faltas **operacionais** (vírus, vermes, *zombies*).

2. Tentativas de intrusão

São faltas **operacionais externas**. O caráter externo não exclui que sejam feitas por operadores ou administradores excedendo seus direitos. Podem usar meios físicos: flutuação de energia, radiação, grampo, aquecimento e resfriamento.

Note a assimetria: o intruso **externo** pode ser um funcionário **interno**. “Externo” aqui é sobre a fronteira do sistema, não sobre o crachá de quem age.

As seis definições de lógica maliciosa

Bomba lógica (<i>logic bomb</i>)	Lógica maliciosa que permanece dormente no sistema hospedeiro até certo instante, evento ou condição, e então apaga arquivos, degrada ou derruba o hospedeiro.
Cavalo de Troia (<i>Trojan horse</i>)	Lógica maliciosa que executa, ou pode executar, uma ação ilegítima dando a impressão de ser legítima. A ação pode ser divulgação ou modificação de informação, ou uma bomba lógica.
Alçapão (<i>trapdoor</i>)	Lógica maliciosa que fornece um meio de contornar mecanismos de controle de acesso.
Vírus (<i>virus</i>)	Lógica maliciosa que se replica e se junta a outro programa quando executada, transformando-o em cavalo de Troia. Pode carregar uma bomba lógica.
Verme (<i>worm</i>)	Lógica maliciosa que se replica e se propaga sem que os usuários percebam. Também pode carregar uma bomba lógica.
Zumbi (<i>zombie</i>)	Lógica maliciosa que pode ser disparada por um atacante para montar um ataque coordenado.

As definições são **composicionais**: vírus vira cavalo de Troia, cavalo de Troia pode conter bomba lógica, verme pode carregar bomba lógica. São peças que se encaixam, não rótulos de catálogo.

“Exploit” traduzido para a taxonomia

O próprio artigo classifica três objetos do dia a dia de segurança. Use isto para calibrar o vocabulário:

Objeto	Classificação	Classes
Invocar um <i>exploit</i>	operacional, externa, humana, software, maliciosa , de interação	24-25
Aquecer a RAM com secador para provocar erros de memória	externa, humana, hardware , maliciosa, de interação	22-23
A vulnerabilidade explorada (ex.: cópia de <i>buffer</i> sem verificação de limite)	desenvolvimento , interna, humana, software, não maliciosa, não deliberada, permanente	1-2

Um *exploit* é, na definição do artigo, um script que exercita uma vulnerabilidade e dá ao invasor acesso, às vezes controle, do sistema.

O ataque e a vulnerabilidade são **faltas distintas**, de classes distintas, criadas por pessoas distintas, em fases distintas do ciclo de vida. Chamar as duas de “o bug” apaga essa estrutura.

Avizienis et al. (2004), §3.2.4

Faltas de interação

- Ocorrem na fase de uso, logo são todas **operacionais**. São causadas por elementos do ambiente de uso interagindo com o sistema, logo são todas **externas**.
- A maioria surge de alguma ação humana, logo são **humanas**. A exceção são as faltas naturais externas (raios cósmicos, tempestades solares), em que a natureza interage com o sistema sem participação humana.

Uma subclasse que domina a operação moderna

Faltas de configuração (*configuration faults*): ajuste errado de parâmetros que afetam segurança, rede, armazenamento, *middleware*. Quando ocorrem durante mudanças de configuração feitas em manutenção adaptativa ou aumentativa concorrente com a operação (por exemplo, subir uma nova versão de software num servidor), são chamadas de **faltas de reconfiguração** (*reconfiguration faults*).

Faltas de interação precisam de uma vulnerabilidade prévia para dar certo. É por isso que endurecer o sistema reduz o efeito de erros de operação, e não apenas o de ataques.

Checkpoint 2

Classifique nas oito dimensões

Um engenheiro de operações, com pressa e sem ler o procedimento, aplica em produção uma regra de *firewall* que ele sabe ser mais permissiva do que a política manda, para destravar um cliente. Dois meses depois, um invasor usa exatamente essa porta para entrar. Classifique

duas faltas separadas: a ação do engenheiro e a ação do invasor.

Checkpoint 2

Classifique nas oito dimensões

Um engenheiro de operações, com pressa e sem ler o procedimento, aplica em produção uma regra de *firewall* que ele sabe ser mais permissiva do que a política manda, para destravar um cliente. Dois meses depois, um invasor usa exatamente essa porta para entrar. Classifique

duas faltas separadas: a ação do engenheiro e a ação do invasor.

A regra permissiva

- operacional (fase de uso)
- externa (vem do ambiente de uso)
- humana
- software
- **não maliciosa**
- **deliberada** (ele sabia)
- acidental ou de incompetência, a decidir
- permanente

Vira uma **vulnerabilidade**: falta interna que habilita uma falta externa.

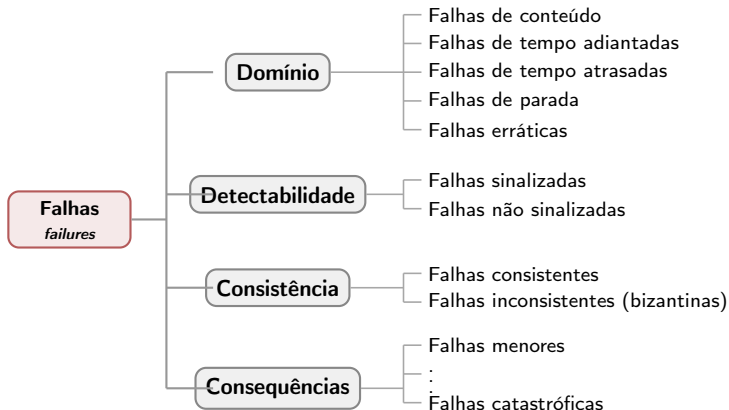
A intrusão

- operacional
- externa
- humana
- software
- **maliciosa**
- deliberada (ver nota)
- não se aplica
- permanente ou transiente, conforme o caso

Sem a primeira falta, a segunda não produz erro.

- 1 Por que este artigo
- 2 Conceitos básicos: sistema, serviço e as três ameaças
- 3 Ameaças I: a taxonomia das faltas
- 4 Ameaças II: falhas, erros e a patologia da falha**
- 5 Atributos revisitados e conceitos vizinhos
- 6 Os meios: prevenção, tolerância, remoção e previsão
- 7 Síntese e guia de leitura

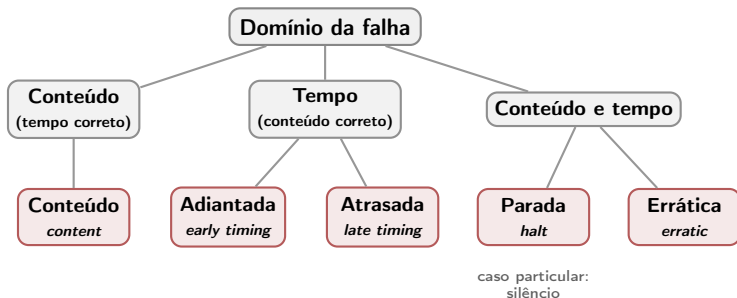
Modos de falha: quatro perguntas sobre uma mesma falha



Toda falha de serviço pode ser descrita respondendo **quatro** perguntas: em que **domínio** ela ocorreu, se foi **sinalizada**, se foi **consistente** entre usuários, e qual a **consequência**.

Avizienis et al. (2004), Fig. 9, §3.3.1

Domínio da falha: o quê e o quando



- **Conteúdo** (*content*): o conteúdo da informação entregue desvia da função. O conteúdo pode ser numérico ou não (alfabetos, gráficos, cores, sons).
- **Tempo** (*timing*): o instante de chegada ou a duração desvia. Pode ser **adiantado** ou **atrasado**.
- **Parada** (*halt*): o estado externo fica constante; nenhuma atividade é perceptível. O caso particular em que **nenhum** serviço é entregue é o **silêncio** (*silence*).
- **Errática** (*erratic*): o serviço é entregue, mas de forma errática. O exemplo do artigo é o *babbling*, o componente que fala sem parar.

Armadilha

Detectabilidade: quem detecta também falha

- A sinalização na interface de serviço vem de mecanismos internos que checam a correção do serviço entregue.
- Se a perda é detectada e sinalizada por um aviso, ocorre uma **falha sinalizada** (*signaled failure*). Caso contrário, **não sinalizada** (*unsigned failure*).

Os dois modos de falha do próprio detector

- 1 sinalizar perda de função quando nenhuma falha ocorreu: **falso alarme** (*false alarm*);
 - 2 não sinalizar uma perda de função que ocorreu: **falha não sinalizada**.
- Quando as falhas resultam em modos reduzidos de serviço, o sistema sinaliza um **modo degradado** ao usuário. Esses modos vão de reduções menores até serviço de emergência e desligamento seguro.

O mecanismo de detecção é ele próprio um sistema, com seus próprios modos de falha. Essa recursão vai reaparecer na §5 sob o nome de **cobertura**.

Consistência: a definição real de “bizantino”

Faz sentido perguntar isso quando o sistema tem **dois ou mais usuários**.

Falha consistente

O serviço incorreto é percebido **identicamente** por todos os usuários.

Falha inconsistente

Alguns ou todos os usuários percebem serviços incorretos **diferentes**. Alguns podem até perceber serviço **correto**. São as chamadas **falhas bizantinas** (*Byzantine failures*).

Armadilha

Bizantino não significa malicioso. Significa **percepções divergentes entre usuários**, por qualquer causa: corrupção de memória, relógio local divergente, caminho de rede diferente. Um nó pode ser bizantino sem que ninguém o tenha atacado.

Se você precisa de acordo entre componentes não falhos sobre um dado de **fonte única** (um sensor local, um relógio local), você caiu no problema do consenso. Voltaremos a ele na §5.

Sistemas de falha controlada

Definição do paper

Sistemas projetados e implementados de modo que falhem **apenas** nos modos descritos na especificação de dependabilidade e segurança, e apenas em extensão aceitável, são **sistemas de falha controlada** (*fail-controlled systems*).

fail-halt / fail-stop	as falhas são, em extensão aceitável, apenas de parada
fail-passive	situação de saída travada (<i>stuck service</i>)
fail-silent	situação de silêncio : nenhum serviço entregue
fail-safe	as falhas são, em extensão aceitável, todas menores

Os três primeiros restringem o **domínio** da falha. O último restringe a **severidade**. São eixos diferentes, e os nomes se parecem demais para o próprio bem.

Na prática

Preferências típicas do artigo: saída travada em vez de valores erráticos; silêncio em vez de *babbling*; falha consistente em vez de inconsistente.

Interrupção, restauração e desligamento

- A entrega de serviço incorreto é uma **interrupção**, que dura até a **restauração do serviço**.
- A duração varia muito conforme a ação de restauração:
 - ① recuperação, reinicialização ou *reboot*, automáticos ou assistidos por operador;
 - ② manutenção corretiva.
- A correção de faltas de desenvolvimento, por correções ou contornos, costuma ser feita **fora de linha**, após a restauração. Os componentes corrigidos entram depois, com ou sem interrupção da operação.

Duas palavras que a operação usa todo dia

Interromper a operação de forma preemptiva, para atualização ou manutenção preventiva, é um **desligamento de serviço**, também chamado **interrupção planejada** (*planned outage*).

A interrupção consecutiva a uma falha é a **interrupção não planejada** (*unplanned outage*).

Janela de manutenção **não é falha**. É parada intencional por entidade autorizada. Isso muda como ela entra no cálculo de disponibilidade.

Falhas de desenvolvimento: o projeto que não nasce

Uma **falha completa de desenvolvimento** encerra o processo antes de o sistema ser aceito para uso. O artigo distingue dois aspectos:

Falha de orçamento

Os fundos alocados se esgotam antes de o sistema passar no teste de aceitação.

Falha de cronograma

A entrega escorrega para um ponto no futuro em que o sistema estaria tecnologicamente obsoleto ou funcionalmente inadequado.

E duas formas **parciais**: **estouro** (*overrun*) de orçamento ou prazo com o desenvolvimento concluído, e **rebaixamento** (*downgrading*), quando o sistema é entregue com menos funcionalidade, menos desempenho, ou dependabilidade prevista menor que a exigida.

Causas principais, segundo o artigo

Especificações incompletas ou faltosas; excesso de mudanças de especificação pedidas pelo usuário; projeto inadequado; faltas de desenvolvimento demais; capacidade insuficiente de remoção de faltas; previsão de dependabilidade insuficiente; estimativas erradas de custo.

Todas, em geral, por subestimar a complexidade do sistema a ser construído.

Falhas de dependabilidade e segurança

Definição do paper

Uma **falha de dependabilidade ou de segurança** ocorre quando o sistema sofre falhas de serviço **mais frequentemente ou mais severamente** do que é aceitável.

- É a diferença entre **falhar** (evento pontual, esperado) e **ser inaceitável** (padrão acumulado ao longo do tempo).
- A especificação explicita as classes de faltas esperadas e o ambiente de uso; pode exigir salvaguardas contra condições perigosas e pode exigir técnicas específicas de prevenção ou tolerância.

Esta é uma das sete contribuições novas de 2004: separar **falha de serviço** de **falha de dependabilidade**. A primeira é um evento. A segunda é um julgamento sobre uma série de eventos.

Na prática

Se você trabalha com objetivos de nível de serviço, já viu isso: um erro isolado não viola o objetivo; violar o orçamento de erro do trimestre, sim.

Erros: latentes, detectados e os que nunca importam

- Um erro é **detectado** (*detected*) se sua presença é indicada por uma mensagem ou sinal de erro. Erros presentes e não detectados são **latentes** (*latent*).
- Quando o erro atinge o estado externo de um **componente**, ocorre falha de serviço **desse componente**, mas o erro permanece interno ao sistema inteiro.

Se um erro vira falha depende de dois fatores

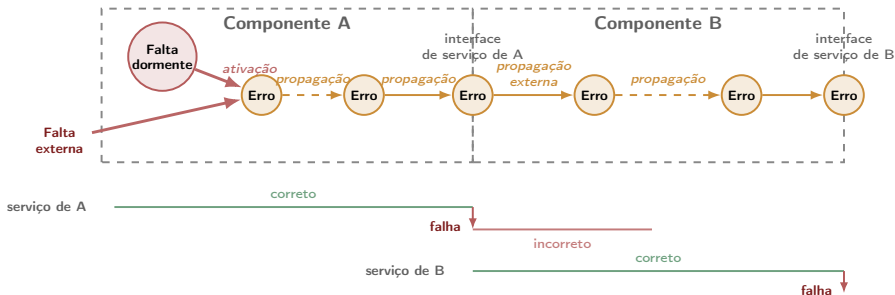
1. A estrutura do sistema, em especial a redundância que existe nele:

- **redundância protetora**, introduzida de propósito para tolerar faltas;
- **redundância não intencional**, porque é difícil ou impossível construir um sistema sem alguma forma de redundância, e ela pode ter o mesmo efeito, de forma inesperada.

2. **O comportamento do sistema**: a parte do estado que contém o erro pode nunca ser necessária para o serviço, ou o erro pode ser eliminado (por sobrescrita) antes de levar a uma falha.

Alguns dos seus sistemas em produção estão funcionando por causa de **redundância não intencional**. É uma sorte que ninguém projetou, e que uma refatoração pode remover sem aviso.

Propagação de erro: dentro e entre componentes

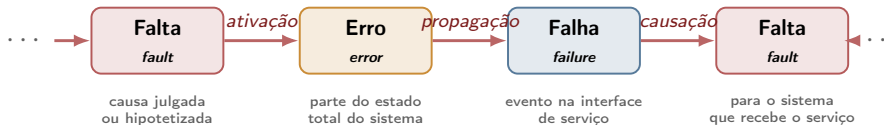


Propagação **interna**: o processo de computação transforma sucessivamente um erro em outros erros.

Propagação **externa**: o erro alcança a interface de serviço de A; o serviço de A para B fica incorreto; essa falha de A aparece como **falta externa** para B e entra por sua interface de uso.

Avizienis et al. (2004), Fig. 10, §3.5

A cadeia fundamental das ameaças



Ativação

Aplicação de uma entrada (o **padrão de ativação**) que faz uma falta dormente ficar ativa.

Propagação

O erro é transformado sucessivamente em outros erros, até alcançar (ou não) a interface de serviço.

Causação

A falha de um componente causa falta, permanente ou transiente, no sistema que o contém, e nos sistemas que recebem seu serviço.

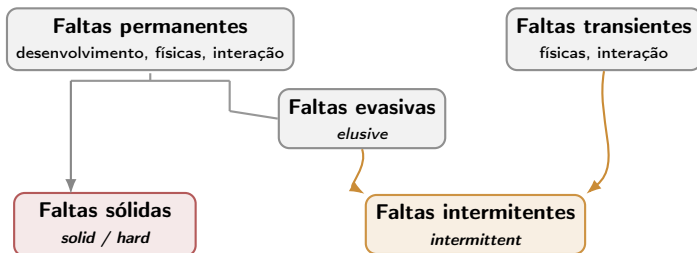
As setas devem ser lidas **genericamente**: por propagação, vários erros podem ser gerados antes de haver falha. A cadeia se instancia por interação entre componentes, por composição de componentes em um sistema, e pela criação ou modificação de um sistema.

Patologia da falha: quatro exemplos do artigo

- 1 Um **curto-circuito** num circuito integrado é uma **falha** do circuito. A consequência (conexão travada em um valor booleano, função alterada) é uma **falta** que fica dormente até ser ativada. Ativada, produz **erro**, que propaga. Se o erro afetar o serviço entregue, ocorre **falha**.
- 2 O erro de um **programador** leva a uma **falha** em escrever a instrução correta, que resulta em **falta dormente** no software. Ativada, produz erro; o erro que afeta o serviço produz falha. **O mesmo vale para uma bomba lógica**, criada por um programador malicioso: dormente até a data marcada, então produz erro que pode causar negação de serviço.
- 3 O erro de um **especificador** leva a uma **falha** em descrever a função, que resulta em **falta na especificação**. O sistema implementado não incorpora a subfunção ausente. Quando os dados de entrada exigem esse serviço, o usuário percebe erro, e ocorre falha.
- 4 Um **alcapão** inserido no sistema é uma **falta de desenvolvimento**, que pode ficar dormente até alguém mal-intencionado usá-lo para entrar. O *login* do intruso é uma **falta de interação deliberada**. Já dentro, ele cria **erro** deliberadamente, por exemplo modificando um arquivo. Quando um usuário autorizado usa esse arquivo, ocorre **falha**.

Repare no padrão que atravessa os quatro: a **falha de um** é a **falta de outro**. A cadeia atravessa fronteiras de sistema, incluindo a fronteira entre pessoas e artefatos.

Reprodutibilidade de ativação: sólidas e evasivas



- **Sólidas** (*solid, hard*): a ativação é reproduzível.
- **Evasivas** (*elusive, soft*): a ativação não é sistematicamente reproduzível.
- **A maior parte das falhas de desenvolvimento residuais em software grande e complexo é evasiva**: as condições de ativação dependem de combinações intrincadas de estado interno e requisições externas, que ocorrem raramente e são difíceis de reproduzir.
- Outros exemplos de falhas evasivas: falhas sensíveis a padrão em memórias semicondutoras; mudanças em parâmetros de componentes de hardware (variação de temperatura, atraso por capacitância parasita); condições que ocorrem só quando a carga do sistema passa de certo nível, causando temporização e sincronização marginais.

Por que agrupar

A manifestação de falhas de desenvolvimento evasivas se parece com a de falhas físicas transitientes. As duas classes são agrupadas como **faltas intermitentes** (*intermittent faults*), e os erros que elas produzem são chamados de **erros suaves** (*soft errors*).



Faltas múltiplas e falhas de modo comum

- **Falta única** (*single fault*): causada por um evento físico adverso ou uma ação humana nociva.
- **Faltas múltiplas**: duas ou mais faltas únicas, concorrentes, sobrepostas ou sequenciais, cujos **erros se sobrepõem no tempo**.

Faltas independentes	atribuídas a causas diferentes; em geral causam erros distintos
Faltas relacionadas	atribuídas a uma causa comum; em geral causam erros similares

Definição do paper

Erros similares são erros que os mecanismos de detecção empregados **não conseguem distinguir**. As falhas causadas por erros similares são **falhas de modo comum** (*common-mode failures*).

Armadilha

As correspondências não são rígidas: faltas independentes, especialmente omissões, podem levar a erros similares, e faltas relacionadas podem levar a erros distintos. O artigo diz “em geral”, e a exceção é exatamente o que derruba redundância.

Checkpoint 3

Classifique os quatro incidentes

Um controlador de nível de tanque publica leituras a cada 100 ms para dois supervisores, S1 e S2.

- (i) Publica 3,2 m quando o nível real é 3,2 m, mas 400 ms atrasado.
- (ii) Para de publicar e acende um LED de erro no painel.
- (iii) Publica 3,2 m para S1 e 8,7 m para S2, sem qualquer aviso.
- (iv) Publica valores plausíveis a 50 Hz em vez de 10 Hz, saturando o barramento.

Classifique cada um dos quatro pontos de vista da Fig. 9. O sistema pode ser chamado de **fail-silent**?

Checkpoint 3

Classifique os quatro incidentes

Um controlador de nível de tanque publica leituras a cada 100 ms para dois supervisores, S1 e S2.

- (i) Publica 3,2 m quando o nível real é 3,2 m, mas 400 ms atrasado.
- (ii) Para de publicar e acende um LED de erro no painel.
- (iii) Publica 3,2 m para S1 e 8,7 m para S2, sem qualquer aviso.
- (iv) Publica valores plausíveis a 50 Hz em vez de 10 Hz, saturando o barramento.

Classifique cada um nos quatro pontos de vista da Fig. 9. O sistema pode ser chamado de **fail-silent**?

	Domínio	Detectab.	Consistência	Consequência
(i)	tempo, atrasada	não sinalizada	consistente	depende da malha
(ii)	parada (silêncio)	sinalizada	consistente	definida pelo projeto
(iii)	conteúdo	não sinalizada	inconsistente (bi-zantina)	potencialmente cat.
(iv)	errática	não sinalizada	consistente	afeta os demais nós

Fail-silent? Não. Os incidentes (i), (iii) e (iv) violam a hipótese. O (iii) contradiz qualquer hipótese de falha controlada. Chamar o sistema de fail-silent seria uma **hipótese de falta** que ninguém garantiu.



- 1 Por que este artigo
- 2 Conceitos básicos: sistema, serviço e as três ameaças
- 3 Ameaças I: a taxonomia das faltas
- 4 Ameaças II: falhas, erros e a patologia da falha
- 5 Atributos revisitados e conceitos vizinhos**
- 6 Os meios: prevenção, tolerância, remoção e previsão
- 7 Síntese e guia de leitura

As definições do artigo diante das normas

Os autores comparam explicitamente sua definição com as que existiam em normas:

ISO	“O termo coletivo usado para descrever o desempenho de disponibilidade e seus fatores de influência: desempenho de confiabilidade, de manutenibilidade e de suporte à manutenção.”
IEC	“A extensão em que se pode confiar que o sistema realiza exclusiva e corretamente as tarefas, sob condições operacionais e ambientais definidas, ao longo de um período ou em um dado instante.”

- A definição da ISO é **centrada em disponibilidade**. Isso se explica pela origem: ela remonta à definição do CCITT, do tempo em que disponibilidade era a preocupação central das operadoras de telefonia.
- A definição da IEC introduz a noção de **confiança depositada** (*reliance*), e por isso está bem mais próxima da do artigo.

Uma norma é um retrato do que preocupava a indústria quando foi escrita. Ler a definição da ISO revela o que era prioridade na telefonia dos anos 1980, não um erro dos seus autores.

Dependência e confiança, com mais precisão

- A relação clássica “**depende de**” da literatura de modularização diz: o componente *a* depende do componente *b* se a correção da entrega de *b* é necessária para a correção da entrega de *a*.
- O artigo aponta a limitação: essa relação é expressa em termos do conceito estreito de **correção**, e por isso é apenas **binária**. A noção de dependência do artigo pode assumir valores num espaço **mensurável**.

Sobre “dependência aceita”

O julgamento de que o nível de dependência é aceitável pode ser explícito, até registrado em contrato. Mas pode ser apenas implícito, ou irrefletido. Pode até ser **involuntário**, quando A não tem outra opção senão depositar sua confiança em B.

“Na medida em que A confia em B, A não precisa assumir a responsabilidade de tolerar as falhas de B. De fato, **a extensão em que A deixa de prover meios de tolerar as falhas de B é uma medida da confiança de A em B.**”

Na prática

Procure no seu código uma chamada externa sem *timeout*, sem *retry* e sem caminho alternativo. Você acabou de encontrar, escrita, a medida da sua confiança naquele serviço.



Nada aqui é absoluto

Definição do paper

A extensão em que um sistema possui os atributos deve ser considerada em sentido **relativo e probabilístico**, não em sentido **absoluto e determinístico**. Por causa da presença ou ocorrência inevitável de faltas, **sistemas nunca são totalmente disponíveis, confiáveis, seguros**.

- Disponibilidade, integridade e manutenibilidade são **geralmente exigidas**, em graus variáveis conforme a aplicação.
- Confiabilidade, segurança física e confidencialidade **podem ou não** ser exigidas, conforme a aplicação.

Se alguém prometer um sistema “seguro” ou “confiável” sem um número e sem uma classe de faltas declarada, a promessa não é verificável. E o que não é verificável não produz confiança **justificada**.

Avizienis et al. (2004), §4.3

Integridade e manutenibilidade vão além do usual

Integridade: “ausência de alterações impróprias do estado do sistema”

As definições usuais falam em **ações autorizadas** e focam em **informação**. A do artigo é mais ampla em três pontos:

- 1 quando o sistema implementa uma política de autorização, “impróprio” naturalmente engloba “não autorizado”;
- 2 “alterações impróprias” englobam ações que **impedem** atualizações corretas da informação;
- 3 “estado do sistema” inclui **modificações e danos ao próprio sistema**, não apenas aos dados.

Manutenibilidade: quatro formas, não duas

A definição vai além de manutenção corretiva e preventiva, e engloba também as formas **adaptativa** (ajuste a mudanças no ambiente) e **augmentativa** (ampliação da função do sistema).

O artigo observa que o objetivo principal da **computação autônoma** (*autonomic computing*) é prover alta manutenibilidade a grandes sistemas em rede, automatizando sua gestão.

Atributos secundários

Além dos seis atributos principais, o artigo admite atributos **secundários**, que refinam ou especializam os primeiros.

Robustez (<i>robustness</i>)	Dependabilidade em relação a faltas externas . Caracteriza a reação do sistema a uma classe específica de faltas.
---------------------------------------	--

Especialmente relevantes para segurança, baseados em distinguir tipos de informação:

Responsabilização (<i>accountability</i>)	Disponibilidade e integridade da identidade de quem realizou uma operação.
Autenticidade (<i>authenticity</i>)	Integridade do conteúdo e da origem de uma mensagem, e possivelmente de outras informações, como o instante de emissão.
Não repúdio (<i>nonrepudiability</i>)	Disponibilidade e integridade da identidade do remetente (não repúdio da origem) ou do destinatário (não repúdio da recepção).

Repare na economia da construção: autenticidade, responsabilização e não repúdio não são conceitos novos. São **combinações** dos atributos básicos aplicadas a tipos específicos de informação.

Política de segurança é especificação

Definição do paper

Uma **política de segurança** (*security policy*) é um conjunto de restrições motivadas por segurança, a serem seguidas por uma organização ou por um sistema. Elas podem ser impostas por controles técnicos, gerenciais ou operacionais.

- O artigo conclui: uma política de segurança é, **na prática, uma especificação parcial do sistema**. Não aderência a ela é considerada uma **falha de segurança**.
- Costuma haver uma **hierarquia** de políticas, refletindo uma hierarquia de sistemas: a empresa, o departamento de TI, os indivíduos e as máquinas do departamento.
- Algumas políticas restringem também **como a informação pode fluir** dentro do sistema, e não apenas os estados permitidos.

Consequência direta: se uma atividade viola a política, a falha de segurança pode ser **do sistema** ou **da política**, que descreveu mal o requisito pretendido. É o mesmo raciocínio das faltas de especificação da §2.2.

Por isso há pesquisa extensa em métodos para expressar e analisar políticas de segurança formalmente. Questões de completude, consistência e exatidão valem aqui como valem para qualquer especificação.

Os atributos entram em conflito

Definição do paper

As variações de ênfase nos diferentes atributos influenciam diretamente o equilíbrio entre as técnicas (prevenção, tolerância, remoção, previsão) a empregar. O problema é agravado porque **alguns atributos são conflitantes**, exigindo compromissos.

Disponibilidade x segurança física

Manter o serviço de pé pode significar operar em condições em que o desligamento seria a opção segura. Um sistema **fail-safe** sacrifica disponibilidade por construção.

Disponibilidade x confidencialidade

Cada réplica, *backup* e *cache* que aumenta disponibilidade é mais uma cópia do dado sensível a proteger.

Aqui está a razão prática de o artigo insistir tanto nos **meios**: eles são muito mais **ortogonais entre si** do que os atributos. Você negocia atributos; você combina meios.

As classes de dependabilidade e segurança são geralmente definidas analisando frequências e severidades de falha e durações de interrupção, diretamente ou por meio de **avaliação de risco**.

Quatro conceitos, o mesmo objetivo

	Dependability	High Confidence	Survivability	Trustworthiness
Objetivo	Capacidade de entregar serviço em que se possa justificadamente confiar; capacidade de evitar falhas mais frequentes ou severas que o aceitável	As consequências do comportamento do sistema são bem compreendidas e previsíveis	Capacidade de o sistema cumprir sua missão de forma tempestiva	Garantia de que o sistema se comportará como esperado
Ameaças tratadas	Faltas de desenvolvimento, faltas físicas, faltas de interação	Ameaças internas e externas; perigos naturais e ataques maliciosos de adversário sofisticado e bem financiado	Ataques (intrusões, sondagens, negação de serviço); falhas (eventos internos); acidentes (eventos externos, como desastres naturais)	Ataques hostis (de invasores ou de pessoal interno); rupturas ambientais; erros humanos e de operador
Origem	Este artigo	<i>Blue Book 2000</i> (EUA)	<i>Survivable network systems</i> (CMU/SEI)	<i>Trust in cyberspace</i>

A conclusão dos autores, após a comparação lado a lado: **os quatro conceitos são essencialmente equivalentes em seus objetivos e tratam ameaças similares**. A diferença está na comunidade que cunhou cada um.

Isso explica a irritação registrada na §1 com a “introdução contínua e desnecessariamente confusa de conceitos supostamente novos para descrever os mesmos meios, atributos e ameaças”.

- 1 Por que este artigo
- 2 Conceitos básicos: sistema, serviço e as três ameaças
- 3 Ameaças I: a taxonomia das faltas
- 4 Ameaças II: falhas, erros e a patologia da falha
- 5 Atributos revisitados e conceitos vizinhos
- 6 Os meios: prevenção, tolerância, remoção e previsão
- 7 Síntese e guia de leitura

Prevenção de faltas

O artigo dedica meia coluna a este meio, com uma justificativa explícita: prevenção de faltas é **parte da engenharia em geral**, e não é específica desta área.

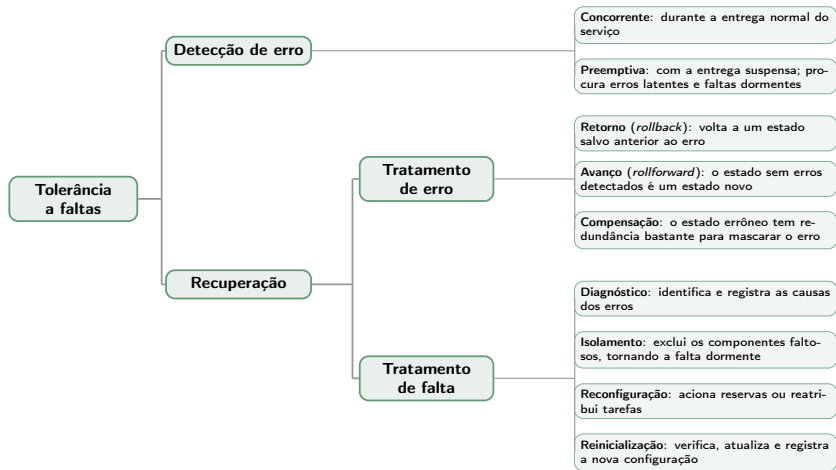
- Prevenção de **faltas de desenvolvimento** é objetivo óbvio das metodologias de desenvolvimento:
 - em software: ocultamento de informação, modularização, linguagens fortemente tipadas;
 - em hardware: regras de projeto.
- A **melhoria dos processos** de desenvolvimento vai um passo além: baseia-se em **registrar as faltas** encontradas nos produtos e eliminar as causas delas por meio de modificações no processo.

Prevenir uma classe de faltas exige saber que ela existe. É por isso que a taxonomia da §3 vem antes: sem ela, “melhorar o processo” vira palpite.

Na prática

Verificador de limites na linguagem, análise estática no *pipeline* e revisão com *checklist* atacam classes de faltas identificáveis. Um treinamento genérico de “qualidade” não ataca classe nenhuma.

Tolerância a faltas: o mapa completo



Tolerância a faltas visa **evitar falhas**, e se realiza por **detecção de erro** e **recuperação do sistema**. Guarde os dois ramos da recuperação: **tratar o erro** limpa o estado; **tratar a falta** impede a próxima ativação.

Detecção de erro: durante ou fora do serviço

Detecção concorrente

Ocorre **durante** a entrega normal do serviço. Exemplos típicos: verificação de soma, códigos detectores de erro, checagens de sanidade, *watch-dog*, comparação entre vias.

Detecção preemptiva

Ocorre **enquanto a entrega normal está suspensa**. Verifica o sistema em busca de **erros latentes** e **faltas dormentes**. Exemplos citados pelo artigo: verificação na energização (*power up*), checagem de reservas (*spare checking*), varredura de memória (*memory scrubbing*), programas de auditoria.

Rejuvenescimento de software (*software rejuvenation*)

Caso particular de detecção e tratamento preemptivos, destinado a **remover os efeitos do envelhecimento de software antes que levem a falha**. Na prática: reiniciar o processo antes que o vazamento de memória o derrube.

A detecção preemptiva é a única que enxerga o que está **dormindo**. Sem ela, você só descobre a falta dormente quando ela acorda, junto com a próxima.



Tratamento de erro: três formas de limpar o estado

Objetivo: **eliminar erros do estado do sistema.**

Retorno (*rollback*) Traz o sistema de volta a um estado salvo que existia **antes** da ocorrência do erro. O estado salvo é o **ponto de recuperação** (*checkpoint*).

Avanço (*rollforward*) O estado sem erros detectados é um estado **novo**. Não se volta ao passado: constrói-se um presente limpo.

Compensação (*compensation*) O estado errôneo contém **redundância suficiente** para permitir que o erro seja **mascarado**.

- Retorno e avanço são invocados **sob demanda**, depois da detecção. A compensação pode ser aplicada sob demanda **ou sistematicamente**, em instantes predeterminados, independentemente da presença de erro detectado.
- Os dois não são mutuamente exclusivos: pode-se tentar retorno primeiro e, se o erro persistir, tentar avanço.

Armadilha

Rollforward não é “tentar de novo seguindo em frente”. É ir para um estado **novo**, aceito como sem erros detectados, abandonando o estado contaminado. Descartar uma amostra corrompida e seguir com a próxima é avanço.



Tratamento de falta: impedir a próxima ativação

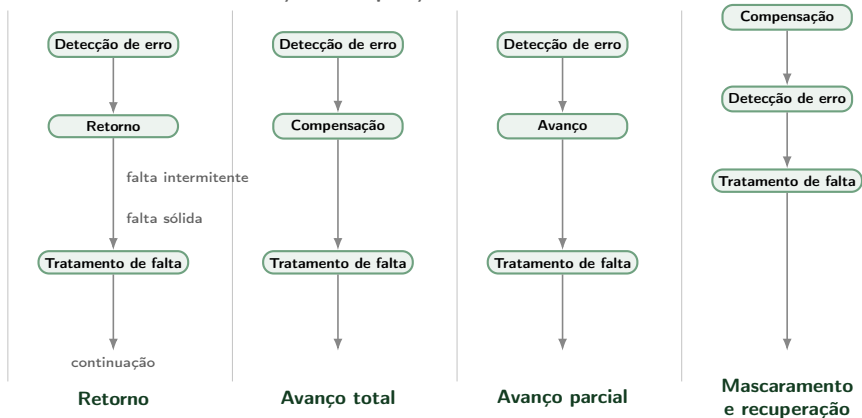
Objetivo: **impedir que as faltas sejam ativadas novamente.**

Diagnóstico (<i>diagnosis</i>)	Identifica e registra as causas dos erros, em termos de localização e tipo .
Isolamento (<i>isolation</i>)	Exclusão física ou lógica dos componentes faltosos de participar da entrega de serviço, ou seja, torna a falta dormente .
Reconfiguração (<i>reconfiguration</i>)	Aciona componentes reservas ou reatribui tarefas entre os componentes não falhos.
Reinicialização (<i>reinitialization</i>)	Verifica, atualiza e registra a nova configuração, e atualiza tabelas e registros do sistema.

- O tratamento de falta costuma ser seguido de **manutenção corretiva**, que remove as faltas isoladas. A distinção continua a mesma: manutenção exige agente externo.
- **Faltas intermitentes não exigem isolamento nem reconfiguração**. Identificar se a falta é intermitente pode ser feito pelo tratamento de erro (a recorrência indica que não é), ou por diagnóstico quando se usa avanço.
- O tratamento de falta pode seguir **diretamente** à detecção, sem tratamento de erro.

As quatro estratégias

Deteccão e recuperação



- **Recuperação do sistema** (*system recovery*) é tratamento de erro sob demanda seguido de tratamento de falta. Daí o nome da estratégia: **deteccão e recuperação**.
- **Mascaramento** (*masking*) resulta do uso **sistemático** da compensação.

O aviso sobre mascaramento

O trecho mais importante da §5.2.1

“O mascaramento vai **ocultar** uma perda de redundância protetora possivelmente progressiva e, no limite, fatal. Por isso, implementações práticas de mascaramento geralmente envolvem detecção de erro (e possivelmente tratamento de falta), levando a **mascaramento e recuperação**.”

- Um votador 2-de-3 silencioso funciona perfeitamente enquanto perde a primeira via, e continua funcionando perfeitamente enquanto perde a segunda.
- O sistema opera sem margem alguma, e ninguém sabe, até a terceira falhar. Aí não há mais o que mascarar.

Mascarar sem detectar troca uma falha visível hoje por uma falha invisível amanhã. Um mecanismo de tolerância que não reporta o que está tolerando é um mecanismo que consome margem em segredo.

Na prática

Todo mecanismo de tolerância deve ter uma métrica de “quantas vezes fui acionado”. Se essa métrica não existe, você não sabe se ele já está no limite.

Hipótese de falta: nada se tolera em abstrato

Definição do paper

A escolha das técnicas de detecção, tratamento de erro e tratamento de falta, e a implementação delas, **dependem diretamente da hipótese de falta** (*fault assumption*) adotada: as classes de faltas que efetivamente podem ser toleradas dependem da hipótese considerada no processo de desenvolvimento. E ela se apóia na **independência das redundâncias** em relação ao processo de criação e ativação de faltas.

Um método bastante usado é realizar **computações múltiplas por canais múltiplos**, em sequência ou concorrentemente.

Faltas físicas

Os canais podem ter **projeto idêntico**, sob a hipótese de que componentes de hardware falham independentemente.

Faltas de desenvolvimento evasivas

Canais idênticos com **retorno** (*rollback*) têm se mostrado adequados, porque a reexecução não reencontra a combinação rara.

Faltas de desenvolvimento sólidas

Canais idênticos **não servem**. Exigem que os canais implementem a mesma função por **projetos e implementações separados**, ou seja, **diversidade de projeto** (*design diversity*).

Componentes autotestáveis, confinamento e consenso

Componente autotestável (*self-checking component*)

Prover, dentro de um componente, a capacidade de processamento funcional exigida **junto com** mecanismos de detecção concorrente de erro. Vale para hardware e para software.

Benefício principal apontado pelo artigo: permite definir com clareza as **áreas de confinamento de erro** (*error confinement areas*).

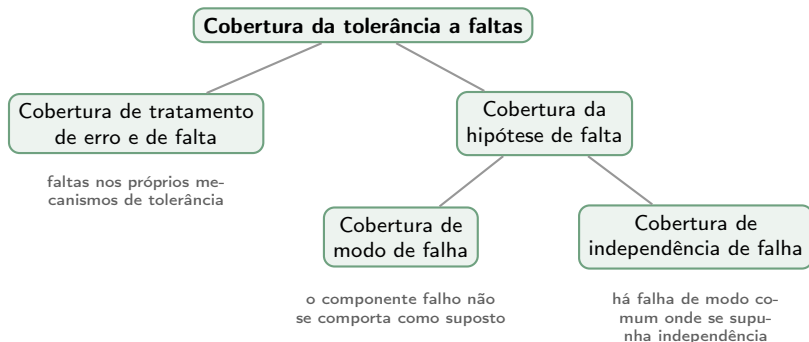
O problema do consenso (*consensus problem*)

Quando um componente precisa comunicar aos demais uma **informação de fonte única** (*single-source information*), como dados de um sensor local, o valor de um relógio local, ou a visão local do estado dos outros componentes, os componentes não falhos precisam **chegar a um acordo** sobre como usar essa informação de forma mutuamente consistente.

Tolerância a faltas é recursiva: os mecanismos que a implementam precisam ser protegidos contra as faltas que podem afetá-los. Exemplos do artigo: replicação do votador, verificadores autotestáveis, memória “estável” para os programas e dados de recuperação.

A introdução sistemática de tolerância costuma ser facilitada por sistemas de apoio dedicados: monitores em software, processadores de serviço, enlaces de comunicação dedicados.

Cobertura: a medida de eficácia, e as três formas de errá-la



- **Cobertura** (*coverage*) é a medida de eficácia de uma dada técnica de tolerância a falhas. As imperfeições da tolerância, ou seja, a **falta de cobertura**, limitam severamente o ganho de dependabilidade possível.
- **Cobertura de tratamento de erro e falha**: faltas de desenvolvimento nos próprios mecanismos de tolerância, em relação às hipóteses declaradas. Definida como **probabilidade condicional** de a técnica ser eficaz, dado que os erros ou faltas ocorreram.
- **Cobertura da hipótese de falha**: as hipóteses diferem das faltas que realmente ocorrem em operação. Se divide em:
 - **cobertura de modo de falha**: os componentes falhos não se

Quando ser conservador piora o sistema

Um resultado contraintuitivo da §5.2.2

Hipóteses de falta conservadoras (por exemplo, supor faltas bizantinas) resultam em **cobertura de modo de falha maior**, às custas de exigir **aumento de redundância** e **mecanismos de tolerância mais complexos**, o que pode levar a uma **diminuição geral** da dependabilidade e da segurança do sistema.

- Mecanismos mais complexos contêm mais faltas de desenvolvimento, o que degrada a **cobertura de tratamento de erro e falta**.
- Você melhora uma espécie de cobertura e piora outra. Não há escolha gratuita.

Um protocolo bizantino mal implementado é pior do que um protocolo simples correto. A hipótese de falta não é opinião: é um requisito que alguém precisa **fazer valer**, por construção do componente ou pagando redundância no protocolo.

Na prática

Alternativa defensável: adotar hipótese fail-silent e **torná-la verdadeira por construção**, com componentes autotestáveis e áreas de confinamento bem definidas. Migrar para hipótese bizantina apenas se o perfil de faltas incluir faltas maliciosas de interação sobre os nós.

Tolerância se aplica a todas as classes de faltas

- A proteção contra intrusões envolve tradicionalmente criptografia e *firewalls*. Alguns mecanismos de detecção de erro servem tanto para faltas não maliciosas quanto para maliciosas, como as técnicas de proteção de acesso à memória.
- A detecção de intrusão é usualmente feita por **checagens de verossimilhança** (*likelihood checks*).
- O artigo lista esquemas propostos para tolerar:
 - **intrusões e faltas físicas**, por fragmentação e dispersão de informação;
 - **lógica maliciosa**, em especial vírus, por verificação de fluxo de controle ou por diversidade de projeto;
 - **intrusões, lógica maliciosa e vulnerabilidades** devidas a faltas físicas ou de desenvolvimento, por diversidade de servidores.

Sinônimos que você vai encontrar por aí

O artigo registra que existem vários sinônimos para tolerância a faltas: **autorreparo** (*self-repair*), **autocura** (*self-healing*) e **resiliência** (*resilience*).

Registra também que **computação orientada à recuperação** (*recovery-oriented computing*) é, em essência, uma abordagem de tolerância a faltas no nível acima do computador individual, em que as falhas dos computadores individuais são as faltas a serem toleradas.



Remoção de faltas durante o desenvolvimento

Três passos: **verificação**, **diagnóstico**, **correção**.

Definição do paper

Verificação (*verification*) é o processo de checar se o sistema adere a propriedades dadas, chamadas **condições de verificação**. Se não adere, é preciso **diagnosticar** as faltas que impediram a adesão e realizar as **correções**.

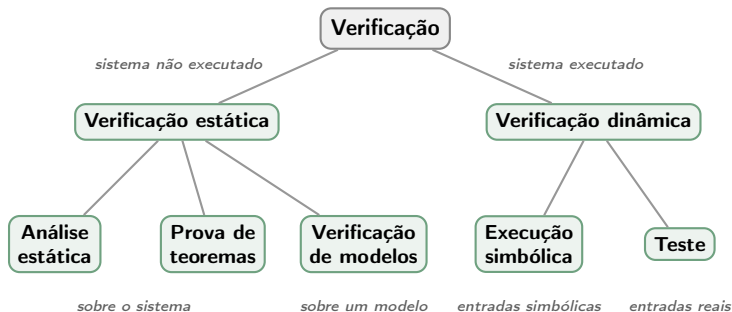
Depois da correção, a verificação deve ser **repetida**, para checar que a remoção não teve consequências indesejadas. Essa etapa é a **verificação de não regressão** (*nonregression verification*).

Validação é outra coisa

Checar a especificação é o que se chama **validação** (*validation*). Faltas de especificação podem ser descobertas em **qualquer** estágio do desenvolvimento: na própria fase de especificação, ou em fases posteriores, quando se encontra evidência de que o sistema não implementará sua função, ou de que a implementação não é viável a custo aceitável.

Verificação pergunta “construí o que o documento manda?”. Validação pergunta “o documento manda a coisa certa?”.

As abordagens de verificação



Estática (sem executar o sistema)

- sobre o **próprio sistema**: análise estática (inspeções, *walk-through*, análise de fluxo de dados, análise de complexidade, interpretação abstrata, checagens do compilador, busca de vulnerabilidades) e **prova de teoremas**;
- sobre um **modelo do comportamento**, em geral um modelo de transição de estados (redes de Petri, autômatos finitos ou infinitos): **verificação de**

Dinâmica (exercitando o sistema)

- entradas **simbólicas**: **execução simbólica** (*symbolic execution*);
- entradas **reais**: **teste** (*testing*).

Armadilha

Model checking não é teste. Ele verifica um **modelo** do comportamento, sem executar o sistema.

Teste: como escolher e como gerar as entradas

Teste exaustivo em relação a todas as entradas possíveis é, em geral, inviável. Os métodos de determinação dos padrões de teste se classificam por dois pontos de vista.

1. Critério de seleção das entradas

Objetivo do teste	conformidade (verificar a capacidade de entregar o serviço especificado) ou busca de faltas
Modelo do sistema	funcional (a partir da função) ou estrutural (a partir da estrutura interna)
Modelo de faltas	baseado em faltas (<i>fault-based</i>) ou baseado em critérios (<i>criteria-based</i>)

2. Geração das entradas

Determinística	Os padrões são predeterminados por escolha seletiva.
Aleatória ou estatística	Os padrões são selecionados segundo uma distribuição de probabilidade definida sobre o domínio de entrada. A distribuição e a quantidade de dados são determinadas conforme o modelo de faltas ou os critérios adotados.

Hardware e software testam por razões diferentes. O teste de hardware visa sobretudo remover **faltas de produção**, e é tipicamente baseado em faltas. O teste de software se



O problema do oráculo, e quatro técnicas nomeadas

Problema do oráculo (*oracle problem*)

Observar as saídas do teste e decidir se elas satisfazem ou não as condições de verificação. Gerar entradas é a parte fácil; decidir se a saída está certa é o problema.

- As condições podem se aplicar a todo o conjunto de saídas ou a uma função compacta delas, como uma **assinatura** do sistema ao testar faltas físicas, ou um **oráculo parcial** ao testar faltas de desenvolvimento de software.
- Ao testar **faltas físicas**, os resultados esperados são obtidos por simulação ou de um sistema de referência, a **unidade padrão** (*golden unit*).
- Ao testar **faltas de desenvolvimento**, a referência é em geral a especificação. Pode também ser um protótipo, ou outra implementação da mesma especificação obtida por diversidade de projeto: **teste costas com costas** (*back-to-back testing*).

As mesmas técnicas se aplicam à verificação dos **próprios mecanismos de tolerância a faltas**, por dois caminhos: verificação estática formal, e teste em que faltas ou erros fazem parte dos padrões de teste, o que se chama **injeção de faltas** (*fault injection*).

Verificar que o sistema **não faz mais do que o especificado** é particularmente importante para segurança física e para segurança, e daí o **teste de penetração** (*penetration testing*).

Remoção de faltas durante o uso

Manutenção corretiva

Remove faltas que **já produziram** um ou mais erros e foram **reportadas**. Para faltas de desenvolvimento, costuma ser feita em estágios: a falta é primeiro **isolada** (por um contorno ou uma correção pontual) antes da remoção efetiva.

Manutenção preventiva

Descobre e remove faltas **antes** que causem erros na operação normal. Alcança:

- 1 faltas físicas ocorridas desde a última ação preventiva;
- 2 **faltas de desenvolvimento que causaram erros em outros sistemas similares.**

- As duas formas se aplicam tanto a sistemas não tolerantes a faltas quanto a sistemas tolerantes.
- Sistemas tolerantes podem ser mantidos **em linha**, sem interromper a entrega de serviço, ou **fora de linha**, durante uma interrupção.

O item 2 é uma das razões de existirem boletins de segurança e listas de *errata*: uma falta que derrubou o sistema de outra pessoa é informação acionável sobre o seu.

Previsão de faltas: duas avaliações

Previsão de faltas é conduzida avaliando o comportamento do sistema em relação à ocorrência ou à ativação de faltas. A avaliação tem dois aspectos.

Qualitativa (ordinal)

Identificar, classificar e ordenar os modos de falha, ou as combinações de eventos (falhas de componentes, condições ambientais) que levariam a falhas do sistema. Método específico citado: **análise de modos de falha e efeitos** (*FMEA*).

Quantitativa (probabilística)

Avaliar, em termos de probabilidades, a extensão em que alguns atributos são satisfeitos. Os atributos passam então a ser vistos como **medidas**. Métodos específicos citados: **cadeias de Markov** e **redes de Petri estocásticas**.

Alguns métodos servem às duas: **diagramas de blocos de confiabilidade** e **árvores de falha**.

Lembrete da §1: os autores declaram que **não há consenso ainda** sobre as **medidas** de dependabilidade e segurança, e que esse ponto exigia elaboração adicional. É o buraco declarado da taxonomia.

Modelagem e teste de avaliação

As duas abordagens principais para obter estimativas probabilísticas são **modelagem** e **teste de avaliação**. Elas são complementares: a modelagem precisa de dados sobre os processos básicos modelados, que podem vir do teste ou do processamento de dados de falha.

- A modelagem tem duas fases: **construir** o modelo do sistema a partir dos processos estocásticos elementares dos componentes (falha, restauração incluindo reparo, ciclo de operação), e **processá-lo** para obter as expressões e os valores das medidas.
- **Modelos de crescimento de confiabilidade** (*reliability growth models*), para hardware, software ou ambos, fazem previsões a partir de dados sobre falhas passadas.
- Medidas relacionadas a desempenho são subsumidas na noção de **performabilidade** (*performability*).

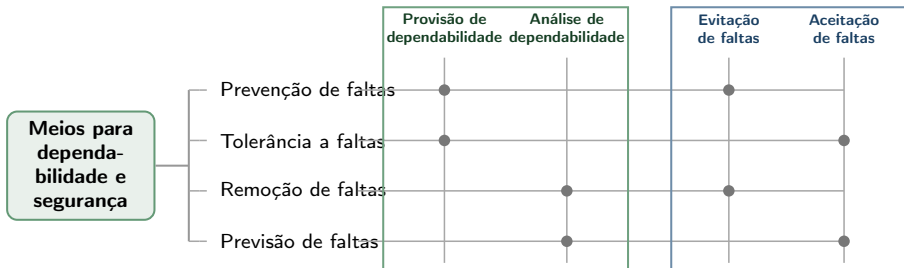
Teste operacional (*operational testing*)

O teste de avaliação é conformidade, funcional, não baseado em faltas, estatístico. A preocupação central é que o **perfil de entrada seja representativo do perfil operacional**. Daí o nome usual.

Ao avaliar sistemas tolerantes a faltas, a **cobertura** dos mecanismos tem influência



As relações entre os quatro meios



Primeiro eixo: o que se obtém

- **Provisão** (prevenção + tolerância): fornece a capacidade de entregar serviço confiável.
- **Análise** (remoção + previsão): alcança **confiança** nessa capacidade.

Segundo eixo: a postura diante da falta

- **Evitação** (prevenção + remoção): buscar sistemas livres de faltas.
- **Aceitação** (tolerância + previsão): conviver com sistemas sujeitos a faltas.

Quando se foca em segurança, a análise de dependabilidade e segurança recebe o nome de **avaliação de segurança** (*security evaluation*).

Por que os quatro meios precisam uns dos outros

O argumento da §5.5 é uma cadeia de imperfeições. Todos os “como fazer” das definições são **objetivos**, raramente alcançados por completo, porque todas as atividades de projeto e análise são atividades **humanas** e, portanto, imperfeitas.

- 1 Apesar da **prevenção** por metodologias e regras de construção (elas próprias imperfeitas, para serem executáveis), faltas ocorrem. Logo, é preciso **remoção**.
- 2 A **remoção** também é imperfeita: nem todas as faltas são encontradas, e novas faltas podem ser introduzidas ao remover uma. Além disso, componentes de prateleira contêm faltas. Logo, é preciso **previsão**.
- 3 A dependência crescente em sistemas computacionais exige **tolerância**, que por sua vez se apóia em regras de construção. Logo, é preciso aplicar **remoção e previsão aos próprios mecanismos de tolerância**.
- 4 E o processo é ainda mais recursivo: os sistemas atuais são tão complexos que seu projeto e implementação precisam de ferramentas de software e hardware. **Essas ferramentas têm elas próprias de ser dependáveis e seguras**, e assim por diante.

É só a **utilização combinada** dos quatro meios, de preferência em cada etapa do projeto e da implementação, que leva a um sistema dependável e seguro.

A avaliação da avaliação

Cobertura, num sentido mais geral

Aqui **cobertura** passa a significar a medida da **representatividade das situações** a que o sistema é submetido durante a análise, comparadas às situações reais que ele enfrentará na vida operacional.

Julgar se um sistema é de fato dependável vai além das técnicas de análise, por três limites:

- 1 Checar com precisão a cobertura das hipóteses de projeto e validação em relação à realidade exigiria domínio da tecnologia e do uso pretendido **muito além do que é geralmente alcançável**.
- 2 Avaliar alguns atributos, em especial segurança diante de certas classes de faltas, é hoje considerado inviável ou pouco significativo, porque as bases probabilísticas **não existem ou não são amplamente aceitas**. Exemplos: segurança física diante de faltas acidentais de desenvolvimento; segurança diante de faltas intencionais.
- 3 As especificações contra as quais a análise é feita **provavelmente contêm faltas**, como qualquer sistema.

Duas consequências práticas: a ênfase em avaliar o **processo** de desenvolvimento (métodos, técnicas, e como foram empregados), às vezes atribuindo uma nota ao sistema;



Checkpoint 4

Classifique cada atividade num meio e nos dois eixos da Fig. 21

- 1 Migrar módulos legados para uma linguagem com tipagem forte e checagem de limites.
- 2 *Model checking* do protocolo de sincronização.
- 3 Derrubar nós ao acaso em homologação para observar a reconfiguração.
- 4 Cadeia de Markov do *cluster*, com taxas de falha e de reparo, para estimar disponibilidade anual.
- 5 Voto majoritário 2-de-3 nos módulos de decisão.
- 6 Rodar a suíte com perfil de entrada representativo da produção e medir taxa de falha.

Checkpoint 4

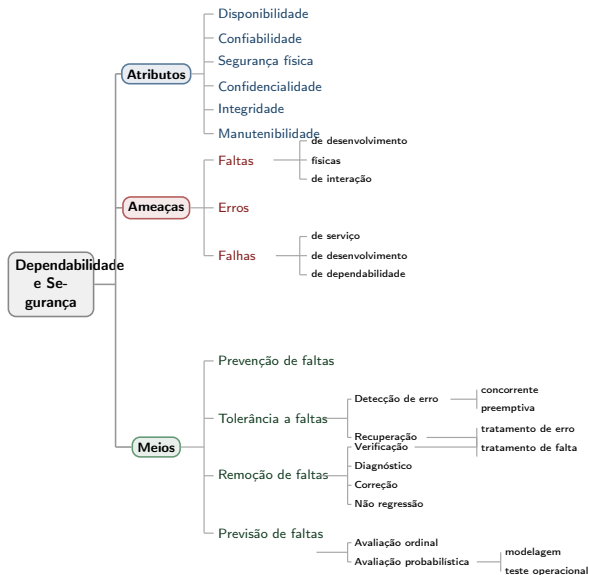
Classifique cada atividade num meio e nos dois eixos da Fig. 21

- ① Migrar módulos legados para uma linguagem com tipagem forte e checagem de limites.
- ② *Model checking* do protocolo de sincronização.
- ③ Derrubar nós ao acaso em homologação para observar a reconfiguração.
- ④ Cadeia de Markov do *cluster*, com taxas de falha e de reparo, para estimar disponibilidade anual.
- ⑤ Voto majoritário 2-de-3 nos módulos de decisão.
- ⑥ Rodar a suíte com perfil de entrada representativo da produção e medir taxa de falha.

Meio	Eixo 1	Eixo 2
1 prevenção de faltas	provisão	evitação
2 remoção (verificação estática sobre modelo)	análise	evitação
3 injeção de faltas, com papel duplo	análise	ver abaixo
4 previsão quantitativa (modelagem)	análise	aceitação
5 tolerância (compensação, portanto mascaramento)	provisão	aceitação
6 previsão por teste operacional	análise	aceitação

- 1 Por que este artigo
- 2 Conceitos básicos: sistema, serviço e as três ameaças
- 3 Ameaças I: a taxonomia das faltas
- 4 Ameaças II: falhas, erros e a patologia da falha
- 5 Atributos revisitados e conceitos vizinhos
- 6 Os meios: prevenção, tolerância, remoção e previsão
- 7 Síntese e guia de leitura

A árvore refinada: a mesma figura, agora com folhas



A taxonomia aplicada a incidentes reais

Quatro casos. Os dois últimos são citados pelo próprio artigo.

Caso	Classe da falta	O que a taxonomia ilumina	Atributo violado
Ariane 5, voo 501 (1996)	desenvolvimento, sólida	Redundância idêntica não protege contra falta idêntica: falha de modo comum	confiabilidade
Therac-25 (1985-87)	desenvolvimento, evasiva	Deteção existia mas a sinalização era inútil; remoção de barreiras físicas	segurança física
<i>Errata</i> de micro-processador	desenvolvimento, permanente	Faltas de desenvolvimento em hardware descobertas após a produção	integridade
Sistema AAS (anos 1990)	incompetência organizacional	Falha completa de desenvolvimento : o projeto não chega ao uso	(nem chegou a operar)

O exercício não é decorar os casos. É perceber que, uma vez classificado o incidente, **o meio adequado quase se escreve sozinho**.

Ariane 5, voo 501: quando duas cópias falham juntas

- **Os fatos.** Em 4 de junho de 1996, o primeiro voo do Ariane 5 foi destruído cerca de 40 segundos após a decolagem. O sistema de referência inercial reaproveitava software do Ariane 4. Uma conversão de um valor de ponto flutuante de 64 bits para inteiro de 16 bits transbordou, porque a trajetória do Ariane 5 produzia valores fora do envelope do Ariane 4. A exceção não era tratada. As **duas** unidades, principal e reserva, rodavam **o mesmo software** e pararam com poucos instantes de diferença. O computador de bordo interpretou dados de diagnóstico como dados de voo.

Falta	Desenvolvimento, interna, humana, software, não maliciosa. Deliberada: a proteção havia sido omitida por decisão de projeto, justificada por margem de carga do processador. Permanente e sólida.
Falta associada	Falta de especificação: a hipótese do envelope de trajetória do Ariane 4 nunca foi revalidada para o Ariane 5.
Erro	O resultado da conversão transbordada no estado da unidade inercial.
Falha	Parada das duas unidades, seguida de conteúdo incorreto entregue ao computador de bordo.

Canais idênticos cobrem falhas físicas independentes. Contra uma falta sólida de de-



Therac-25: detecção sem sinalização útil

- **Os fatos.** Entre 1985 e 1987, o acelerador de radioterapia Therac-25 aplicou doses de radiação muito acima da prescrita em pelo menos seis acidentes, com mortes. O modelo anterior tinha **intertravamentos físicos** que impediam a configuração perigosa; no Therac-25 essa proteção passou a ser feita apenas por software. Uma condição de corrida, ativada quando o operador editava a tela de comando com rapidez incomum, deixava a máquina em estado inconsistente. O sistema exibia mensagens de erro numéricas sem significado documentado, e o procedimento usual era simplesmente mandar continuar.

A cadeia

Falta: condição de corrida, de desenvolvimento, **evasiva** (ativação dependia de temporização rara e não era reproduzível em bancada).

Erro: estado de configuração inconsistente.

Falha: conteúdo incorreto de dose, com severidade **catastrófica**.

O que a taxonomia expõe

A detecção **existia**: a falha era **sinalizada**. Mas a sinalização não permitia ao operador distinguir condição perigosa de ruído, e o efeito prático foi o de uma falha **não sinalizada**.

Remover os intertravamentos trocou uma barreira de **prevenção** por uma promessa de **correção** em software, sem a cobertura correspondente.

Dois casos que o próprio artigo usa

Errata de microprocessador

O artigo cita como classe de faltas as *errata*: faltas de desenvolvimento em hardware descobertas **depois** do início da produção, publicadas em atualizações de especificação ao longo de toda a vida do processador. O caso público mais conhecido é o da divisão em ponto flutuante de um processador de 1994, cuja tabela de consulta incompleta produzia resultados incorretos em divisões raras, e cujo programa de substituição custou centenas de milhões de dólares ao fabricante. **Leitura taxonômica**: falta de desenvolvimento, interna, humana, **hardware**, não maliciosa, não deliberada, permanente, **sólida**. Atributo violado: **integridade** do resultado. Meio aplicável depois de descoberta: **remoção** (recall) ou **tolerância** (contorno em software).

O sistema AAS de controle de tráfego aéreo

Citado duas vezes no artigo. Primeiro como exemplo de **incompetência organizacional**: uma equipe ou organização inteira sem condições de fazer o trabalho. Depois como exemplo de **falha completa de desenvolvimento**, com desperdício de US\$ 1,5 bilhão. E ainda como exemplo de **falta na própria especificação de dependabilidade**: o limite inicial de 3 segundos de indisponibilidade por ano foi renegociado para 5 minutos por ano em 1994. **Leitura taxonômica**: o sistema nunca entrou em uso, logo não houve falha de serviço. Houve falha de **desenvolvimento**, nas duas formas: orçamento e cronograma.

Usando a taxonomia num projeto real

Um roteiro de sete perguntas, na ordem em que o artigo as constrói:

- 1 **Onde está a fronteira do sistema?** Escreva em uma frase. Depois mova-a para incluir o operador humano e veja o que muda de classificação.
- 2 **Qual é a função, e onde está a especificação?** Se não houver especificação escrita, ainda assim há função, e falhas continuarão ocorrendo em relação a ela.
- 3 **Quais atributos importam, e em que grau?** Nomeie os que **não** importam. E identifique os conflitos entre os que importam.
- 4 **Quais classes de faltas estão na especificação?** Use as oito dimensões. O que ficou de fora é o que você decidiu não tolerar.
- 5 **Quais modos de falha são aceitáveis?** Domínio, detectabilidade, consistência, severidade. Isso define se o sistema é fail-silent, fail-safe, ou nenhum dos dois.
- 6 **Que hipóteses de falta estão implícitas no seu código?** Liste três que nunca foram escritas. Para cada uma, diga que espécie de cobertura cai se ela for falsa.
- 7 **Os quatro meios estão equilibrados?** Coloque suas atividades nos quadrantes da Fig. 21. O quadrante vazio é o seu risco.

A taxonomia não é um *checklist* de conformidade. É um instrumento para descobrir o que você assumiu sem perceber.

Como ler o artigo: três passadas, fora de ordem

Passada 1, cerca de 60 min: o esqueleto

- 1 **§2 inteira** (p. 12-14). Devagar, com lápis. É o vocabulário de todo o resto. Ao terminar, desenhe de memória o par provedor-usuário com as duas interfaces.
- 2 **§3.5 e Fig. 10, 11, 12** (p. 20-21). **Fora de ordem, de propósito.** A patologia fecha o entendimento da tríade antes de você enfrentar a taxonomia de faltas.
- 3 **Fig. 2** (p. 14). Guarde como mapa e volte a ela ao fim de cada seção.

Passada 2, cerca de 90 min: as três grandes classificações

§3.1 como *checklist* de projeto. **§3.2 com a Fig. 4 aberta ao lado**; depois §3.2.2 a §3.2.5 com a Fig. 5. **Não decore os 31 números**: o que importa são os oito critérios, os três agrupamentos e o fato de que nem todo critério se aplica. **§3.3 com as Fig. 8 e 9.** **§3.4**, curta. **§5.2 com as Fig. 16, 17 e 18**: é a seção mais aplicada do artigo.

Passada 3, cerca de 60 min: completar e comparar

§4.1 e §4.2: o “porquê” da definição alternativa e do par dependência/confiança. **§5.3 e §5.4** com as Fig. 19 e 20; a §5.3.1 merece leitura cuidadosa até o problema do oráculo. **§5.5 e §6** com as Fig. 21 e 22.

Cinco perguntas para responder enquanto você lê

Responda por escrito, no máximo cinco linhas cada, usando um sistema que você conheça: um projeto da disciplina, um estágio, um serviço que você usa todo dia.

- 1 **Onde está a fronteira?** Escreva a fronteira em uma frase. Depois mova-a para incluir o operador humano e liste duas faltas que mudaram de classificação.
- 2 **Qual erro nunca virou falha?** Descreva um caso, do seu próprio código, em que houve erro no estado e nenhuma falha de serviço. Qual dos dois fatores da §3.4 explica isso: a estrutura ou o comportamento?
- 3 **Qual hipótese de falta o seu sistema assume sem ter escrito?** Liste três. Para cada uma, diga que espécie de cobertura da Fig. 18 está em jogo se ela for falsa.
- 4 **Onde a sua confiança está codificada?** Ache um ponto em que você depende de outro sistema e não trata a falha dele. Segundo a §4.2, essa é a medida da sua confiança. Você assinaria essa medida?
- 5 **Os quatro meios estão equilibrados?** Liste as atividades de qualidade do seu projeto nos quadrantes da Fig. 21. Qual está vazio, e o que a §5.5 diz sobre isso?

Exercícios (1/3): classificar nas oito dimensões

E1. Um programador escreveu, sem perceber, uma cópia de *buffer* sem verificação de limite em um serviço de rede. Dois anos depois, um atacante externo dispara um script que explora essa cópia e obtém acesso privilegiado na máquina.

Classifique **as duas** faltas envolvidas nos oito critérios da Fig. 4, e diga o grupo de cada uma. **E2.** Compare dois casos de hardware que parecem iguais:

- (a) uma solda fria não detectada na linha de montagem deixa uma trilha em curto desde o primeiro dia de uso;
- (b) durante a operação, variação de temperatura altera o atraso de propagação de um bloco combinacional, e o circuito só viola o tempo de estabelecimento quando a carga passa de 80%.

Classifique os dois nas oito dimensões. Qual é sólida e qual é evasiva? Que consequência isso tem para a estratégia de teste? **E3.** Para fechar um chamado urgente, um administrador desabilita a verificação de certificado em um cliente de integração e esquece de reverter. Ele sabia que estava enfraquecendo a verificação e julgou o risco aceitável. Seis meses depois, um ataque de intermediário lê o tráfego dessa integração. Classifique a ação do administrador nas oito dimensões, diga o grupo, e explique o papel dela na falha de confidencialidade.

Exercícios (2/3): cadeia, modos de falha e decisão de projeto

E4. Um microserviço de pagamento P consome um serviço de câmbio C. C tem um arredondamento errado que produz cotações com duas casas a menos em moedas de baixa denominação. P usa a cotação, grava o valor convertido no banco e devolve um recibo ao cliente final.

Desenhe a cadeia falta → erro → falha para C e para P. Marque onde há propagação interna e onde há propagação externa, e indique em que momento existe erro latente.

E5. Volte ao Checkpoint 3 (controlador de nível de tanque). Escreva a **especificação de dependabilidade** que tornaria o sistema aceitável: quais modos de falha você admite, com que severidade máxima, e sob que classes de faltas. Depois diga que mecanismo de detecção cada admissão exige. **E6.** Você projeta a camada de tolerância de um

controlador de voo. O perfil de faltas esperado tem três componentes: (a) faltas físicas transientes por radiação; (b) faltas de desenvolvimento evasivas, ativadas por combinações raras de estado e carga; (c) faltas de desenvolvimento **sólidas** no algoritmo de controle.

Escolha, para cada componente, entre **replicação de canais idênticos com retorno** e **diversidade de projeto com comparação**. Justifique pela §5.2.2 e discuta o custo em termos de cobertura.

Exercícios (3/3): hipóteses e equilíbrio dos meios

E7. Um *cluster* de cinco nós replica um serviço de estado. O arquiteto propõe adotar hipótese de falta bizantina com o protocolo de acordo correspondente. Um colega propõe hipótese fail-silent, com detecção por *heartbeat* e um protocolo bem mais simples.

Argumente pelos dois lados usando **hipótese de falta, cobertura de modo de falha e complexidade**. Diga o que você decidiria e **sob que condição** mudaria de ideia. **E8.** Liste as atividades de qualidade de um projeto que você conheça e classifique cada uma:

- 1 em um dos **quatro meios** (§2.4);
- 2 nos **dois eixos** da Fig. 21 (provisão ou análise; evitação ou aceitação).

Depois responda: **(a)** há algum quadrante vazio? **(b)** o que a §5.5 diz sobre a consequência de deixar um quadrante vazio?

Em E1, E3 e E7 há mais de uma resposta defensável. O que se avalia é a **justificativa pela taxonomia**, não a coincidência com um gabarito. Em E1, registre também a divergência entre a §3.2.4 e a Fig. 6 quanto à intenção das faltas maliciosas.

O que fica

- ❶ **Falha é desvio do serviço em relação à função**, não em relação ao documento. O documento também pode estar errado.
- ❷ **A cadeia atravessa fronteiras**: a falha de um é a falta de outro. Toda a taxonomia é relativa à fronteira que você desenhou.
- ❸ **Muitos erros nunca viram falha**. Estrutura e comportamento decidem, e parte disso é sorte que ninguém projetou.
- ❹ **Segurança não é um campo separado**: um ataque é uma falta externa maliciosa, e segurança é o que sobra dos atributos quando se acrescenta a palavra “autorizado”.
- ❺ **Você nunca tolera faltas em abstrato**: tolera uma hipótese declarada, e paga em cobertura quando ela é falsa.
- ❻ **Os quatro meios são imperfeitos e se sustentam mutuamente**. Dois fornecem a capacidade, dois fornecem a justificativa.

A §1 termina antecipando que novos desafios virão de forma inesperada, “talvez como as chamadas propriedades emergentes, como as do computador HAL de *2001: Uma Odisseia no Espaço*”, à medida que a complexidade dos sistemas homem-máquina que conseguimos construir exceder nossa capacidade de compreendê-los. Em 2004, isso era uma nota de rodapé.

Referências

- A. Avizienis, J.-C. Laprie, B. Randell, C. Landwehr. *Basic Concepts and Taxonomy of Dependable and Secure Computing*. IEEE Transactions on Dependable and Secure Computing, v. 1, n. 1, p. 11-33, jan-mar 2004.
<https://www.landwehr.org/2004-aviz-laprie-randell.pdf>
- Referências internas citadas nesta aula, todas da lista do próprio artigo:
 - 3 A. Avizienis. *Design of Fault-Tolerant Computers*, 1967.
 - 4 , [6] Avizienis et al., sobre programação em N versões e diversidade de projeto, 1977 e 1984.
 - 5 Avizienis & He. *Microprocessor Entomology: A Taxonomy of Design Faults in COTS Microprocessors*, 1999.
 - 27 Huang et al. *Software Rejuvenation*, 1995.
 - 38 Lamport, Shostak & Pease. *The Byzantine Generals Problem*, 1982.
 - 39 Landwehr et al. *A Taxonomy of Computer Program Security Flaws*, 1994.
 - 40 , [41] Laprie: a síntese do FTCS-15 (1985) e o livro *Dependability: Basic Concepts and Terminology* (1992).
 - 67 Análise da falha completa de desenvolvimento do sistema AAS.

O apêndice do artigo traz um **índice de definições** com a seção em que cada termo é definido. Use-o como dicionário enquanto lê.